FonoKids · funciones y clases con su código completo

Documento para estudiar el proyecto o para pasárselo a una IA como contexto. Contiene **todas las funciones y clases** del código que usa la aplicación. completas y sin recortar. en el orden del pipeline. Total: **108 funciones**. **20 clases** y 3.716 líneas de código.

Qué es el proyecto

FonoKids es una aplicación web para que niños practiquen la pronunciación de sonidos difíciles del español (C/K, G suave, sinfones, S y RR fuerte). Evalúa la voz y el movimiento de la cara con modelos de IA y responde con mensajes y estrellas, sin porcentajes. Servidor en FastAPI (Python/PyTorch) y aplicación en React + Vite.

Cómo funciona

- **Video (cara completa):** HRNet/FAN detecta 68 puntos de la cara por frame. Se normalizan con las esquinas externas de los ojos (puntos 36 y 45) → 136 números; se suavizan y se les agrega velocidad y aceleración → 408 números por frame; se dejan en 60 frames. La red `LipReadingConformer` (TCN de 3 bloques + Conformer de 2 capas, 8 cabezas, dimensión 128; el nombre quedó de cuando usaba solo labios) clasifica en 54 clases (palabra × correcto/incorrecto). Resultado en validación: 89,67 % exacto, 99,17 % Top-3, 90,36 % bien o mal, con 10.917 clips.
- **Audio:** wav2vec2 (facebook/wav2vec2-xlsr-53-espeak-cv-ft, modelo externo) da fonemas. GOP = puntuación por fonema con alineación forzada (fonema sospechoso si el puntaje es menor que -3.0). Además un clasificador entrenado (`AudioClassifierHead`, 58 clases) sobre el embedding de 1024 números, 97,4 % de acierto.
- **Decisión:** el audio decide cuando está seguro (probabilidad ≥ 0.60, o ≥ 0.30 con el GOP bien); el video solo bloquea cuando está seguro de que está mal (≥ 0.75). Si la palabra no tiene modelo de video (hoy, `perro`) se evalúa solo con audio.
- **Ejercicios de calentamiento:** praxias con cámara, soplo con micrófono, pa-ta-ka y trabalenguas, con estimación de esfuerzo en estrellas (versión de prueba).

Notas: los cuadernos de Colab muestran el código por celda. Los scripts antiguos del modelo de solo labios no se incluyen porque ese modelo quedó desactivado; en el servidor las líneas con la marca `[LABIOS DESACTIVADO]` están comentadas. Las pantallas de React (carpeta `web/src/screens`) no se incluyen; sí el código de ejercicios, de llamadas al servidor y del grabador.

Índice

1. Colab · Extraer landmarks de la cara completa

def instalar_si_falta
Imports, constantes y código suelto
def buscar_raiz_local
Imports, constantes y código suelto
def clase_del_video
Imports, constantes y código suelto
def build_hrnet_detector
def detect_hrnet
def normalize_face_landmarks
def is_outlier
def fill_missing_frames
def smooth_positions
def add_dynamics
def process_clip
Imports, constantes y código suelto
Imports, constantes y código suelto
Imports, constantes y código suelto

2. Colab · Entrenar el modelo de video (cara completa)

def instalar_si_falta
Imports, constantes y código suelto
def buscar_raiz_local
Imports, constantes y código suelto
def _nfc
class LipReadingDataset
def build_dataloaders
class PositionalEncoding
class TCNBlock
class TCNFeatureExtractor
class ConformerFeedForward
class ConformerConvModule
class ConformerBlock
class ConformerEncoder
class LipReadingConformer
def masked_mean_pool
def make_padding_mask
class EarlyStopping
def run_epoch
def build_warmup_cosine_scheduler
Imports, constantes y código suelto
Imports, constantes y código suelto
Imports, constantes y código suelto
Imports, constantes y código suelto

3. Audio · fonemas, GOP y embeddings (scripts/audio_pronunciation.py)

def texto_a_fonemas
def load_model
def load_audio_16k
def extract_embedding
def _phoneme_to_token_ids
def _greedy_ctc_decode
def fonemas_a_texto_aproximado
def _distancia_edicion
def _align_ops
def diagnose_tipo_error
def score_pronunciation

def _localizar_subsecuencia
def score_word_in_sentence
def print_report
Imports, constantes y código suelto

4. Audio · entrenamiento del clasificador (scripts/train_audio_classifier.py)

def find_labeled_clips
def build_embeddings
def stratified_split
class AudioClassifierHead
class EarlyStopping
def plot_training_curves
def plot_confusion_matrix
def main
Imports, constantes y código suelto

5. Servidor FastAPI (server/main.py)

def _parse_clase_predicha
def build_hrnet_detector
def detect_landmarks_batch
def normalize_lip_landmarks
def normalize_face_landmarks
def is_outlier
def fill_missing_frames
def smooth_positions
def add_dynamics
class PositionalEncoding
class TCNBlock
class TCNFeatureExtractor
class ConformerFeedForward
class ConformerConvModule
class ConformerBlock
class ConformerEncoder
def masked_mean_pool
class LipReadingConformer
def make_padding_mask
def safe_torch_load
def load_everything
def cleanup
def preprocess_sequence
def health
def _count_files
def dataset_stats
def _parse_clip_filename
def dataset_recordings
def dataset_thumbnail
def _pick_reference_clip
def reference_landmarks
def _guardar_debug_frase
def _save_practice_clip
def _score_audio_pronunciation
def _score_word_in_sentence
def _classify_audio
def _run_audio_only_pipeline
def _run_prediction_pipeline
def predict
def _distancia_frase
def predict_frase
def _ciclos_pataka
def _palabras_flojas
def ejercicio_resultado
def ejercicios_resumen
def ejercicio_voz
Imports, constantes y código suelto

6. Base de datos (server/db.py)

def _get_pool
def init_db
def save_attempt
def save_attempt_at
def get_recordings_grouped
def save_ejercicio
def get_ejercicios_resumen
Imports, constantes y código suelto

7. Web · ejercicios y esfuerzo (web/src/ejercicios.js)

function media
function nivel
function esfuerzoPraxia
function esfuerzoSoplo
function esfuerzoVoz
Imports, constantes y código suelto

8. Web · llamadas al servidor (web/src/api.js)

function normalize
function getServerUrl
function thumbnailUrl
function setServerUrl
function withTimeout
function checkHealth
function getHealth
function getReferenceLandmarks
function wordsFromClasses
function getDatasetStats
function getRecordings
function waitForServerBack
function predictOnce
function predictFrase
function predict
function ejercicioVoz
function guardarEjercicio
Imports, constantes y código suelto

9. Web · grabador de audio (web/src/audioRecorder.js)

function encodeWav
function createAudioRecorder
Imports, constantes y código suelto

1. Colab · Extraer landmarks de la cara completa

def instalar_si_falta

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 2

```
def instalar_si_falta(modulo, paquete):
    try:
        __import__(modulo)
    except ImportError:
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', paquete])
```

Imports, constantes y código suelto

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 2

```
import os, re, sys, json, math, glob, subprocess

try:
    import google.colab
    EN_COLAB = True
except ImportError:
    EN_COLAB = False
print('Corriendo en:', 'Google Colab' if EN_COLAB else 'tu PC (local)')

if EN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')

import torch
print('CUDA disponible:', torch.cuda.is_available(), '(en Colab: Entorno de ejecucion > Cambiar tipo > GPU)')

instalar_si_falta('face_alignment', 'face-alignment')
import face_alignment
import cv2
import numpy as np
```

def buscar_raiz_local

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 4

```
def buscar_raiz_local():
    actual = os.path.abspath(os.getcwd())
    for _ in range(5):
        if os.path.isdir(os.path.join(actual, 'data')) and os.path.isdir(os.path.join(actual, 'scripts')):
            return actual
        actual = os.path.dirname(actual)
    return r'D:\FonoKids'
```

Imports, constantes y código suelto

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 4

```
RUTA_VIDEOS = '/content/drive/MyDrive/Aplicaciones_Inteligentes/SACA LANDMARKS/datos/sesiones_continuas'

if not os.path.isdir(RUTA_VIDEOS):
    RUTA_VIDEOS = os.path.join(buscar_raiz_local(), 'data', 'sesiones_continuas')
RUTA_VIDEOS = RUTA_VIDEOS.rstrip('/\\')
CARPETA_SALIDA = os.path.dirname(RUTA_VIDEOS)
assert os.path.isdir(RUTA_VIDEOS), f'No encontre {RUTA_VIDEOS}: cambia RUTA_VIDEOS por la carpeta de tu Drive que tiene los videos'
print('Videos en:     ', RUTA_VIDEOS)
print('Resultados en: ', CARPETA_SALIDA)

SOLO_CLASES = None
EXTENSIONES = ('.avi', '.webm')
MAX_CLIPS_POR_CLASE = None
MIN_FRACCION_CON_CARA = 0.5

CARPETAS_VIDEOS = [RUTA_VIDEOS]
_archivada = os.path.join(os.path.dirname(CARPETA_SALIDA), 'dataARCHIVADO')
if os.path.isdir(_archivada):
    CARPETAS_VIDEOS.append(_archivada)

LANDMARKS_OUT_DIR = os.path.join(CARPETA_SALIDA, 'landmarks_cara_completa_npy')
_viejos = [os.path.join(RUTA_VIDEOS, 'data', 'landmarks_npy'), os.path.join(CARPETA_SALIDA, 'landmarks_npy')]
LANDMARKS_VIEJOS_DIR = next((c for c in _viejos if os.path.isdir(c)), _viejos[0])
LABEL_MAP_PATH = os.path.join(LANDMARKS_OUT_DIR, 'label_map.json')
os.makedirs(LANDMARKS_OUT_DIR, exist_ok=True)
```

def clase_del_video

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 6

```
def clase_del_video(ruta):
    carpeta = os.path.basename(os.path.dirname(ruta))
    if carpeta.lower() != 'clips':
        return None
    return os.path.basename(os.path.dirname(os.path.dirname(ruta)))
```

Imports, constantes y código suelto

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 6

```
videos_por_clase = {}
for raiz in CARPETAS_VIDEOS:
    if not os.path.isdir(raiz):
        print(f'(no existe, se omite) {raiz}')
        continue
    for carpeta, _, archivos in os.walk(raiz):
        for f in archivos:
            if f.lower().endswith(EXTENSIONES):
                ruta = os.path.join(carpeta, f)
                clase = clase_del_video(ruta)
                if clase and (SOLO_CLASES is None or clase in SOLO_CLASES):
                    videos_por_clase.setdefault(clase, {}).setdefault(os.path.splitext(f)[0], ruta)

clases = sorted(videos_por_clase)
assert clases, f'No encontre videos {EXTENSIONES} en <clase>/clips/ dentro de {RUTA_VIDEOS}'

print(f"{'clase':34s} {'videos':>7s} {'landmarks viejos (boca)':>25s}")
total_videos, total_viejos = 0, 0
for c in clases:
    n_v = len(videos_por_clase[c])
    viejos_dir = os.path.join(LANDMARKS_VIEJOS_DIR, c)
    n_o = len([f for f in os.listdir(viejos_dir) if f.endswith('.npy')]) if os.path.isdir(viejos_dir) else 0
    total_videos += n_v; total_viejos += n_o
    marca = '' if n_o == 0 or n_v >= n_o else '   <-- faltan videos'
    print(f'{c:34s} {n_v:7d} {n_o:25d}{marca}')
print(f"{'TOTAL':34s} {total_videos:7d} {total_viejos:25d}")
print(f'{len(clases)} clases con videos.')
if total_viejos and total_videos < total_viejos:
    print(f'\nOJO: hay {total_videos} videos para {total_viejos} landmarks viejos. Con la cara completa solo se pueden '
          f'procesar los clips cuyo VIDEO original existe: sube el resto de los videos antes de continuar si quieres usar todos los
datos.')
```

def build_hrnet_detector

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
def build_hrnet_detector():
    try:
        import torch._dynamo
        torch._dynamo.config.suppress_errors = True
    except Exception:
        pass
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    if device == 'cpu':
        print('[AVISO] No hay GPU: va a funcionar, pero mas lento.')
    detector = face_alignment.FaceAlignment(
        face_alignment.LandmarksType.TWO_D, device=device, flip_input=False, compile=False,
    )
    print(f'>>> Detector HRNet/FAN corriendo en {device.upper()}. <<<')
    return detector
```

def detect_hrnet

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
def detect_hrnet(landmarker, frame_bgr):
    rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
    preds = landmarker.get_landmarks(rgb)
    if not preds:
        return None
    return preds[0].astype(np.float32)
```

def normalize_face_landmarks

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
def normalize_face_landmarks(landmarks_px):
    right_eye = landmarks_px[EYE_OUTER_RIGHT].astype(np.float64)
    left_eye = landmarks_px[EYE_OUTER_LEFT].astype(np.float64)
    center = (right_eye + left_eye) / 2.0
```

```
    eye_vector = left_eye - right_eye
    scale = np.linalg.norm(eye_vector)
    if scale < 1e-3:
        return None
    angle = math.atan2(eye_vector[1], eye_vector[0])
    cos_a, sin_a = math.cos(-angle), math.sin(-angle)
    rotation = np.array([[cos_a, -sin_a], [sin_a, cos_a]], dtype=np.float64)
    pts = landmarks_px[FACE_INDICES].astype(np.float64)
    normalized = ((pts - center) @ rotation.T) / (scale + SCALE_EPSILON)
    return normalized.astype(np.float32).flatten()
```

def is_outlier

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
def is_outlier(vector, last_valid_vector):
    if last_valid_vector is None:
        return False
    return float(np.abs(vector - last_valid_vector).max()) > OUTLIER_MAX_JUMP
```

def fill_missing_frames

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
def fill_missing_frames(positions):
    validos = [i for i, p in enumerate(positions) if p is not None]
    if not validos:
        return None
    primero, ultimo = validos[0], validos[-1]
    relleno = list(positions)
    for i in range(primero):
        relleno[i] = relleno[primero]
    i = primero
    while i < ultimo:
        if relleno[i + 1] is None:
            j = i + 1
            while relleno[j] is None:
                j += 1
            inicio, fin = relleno[i], relleno[j]
            pasos = j - i
            for k in range(1, pasos):
                t = k / pasos
                relleno[i + k] = (1 - t) * inicio + t * fin
            i = j
        else:
            i += 1
    for i in range(ultimo + 1, len(relleno)):
        relleno[i] = relleno[ultimo]
    return np.stack(relleno, axis=0).astype(np.float32)
```

def smooth_positions

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
def smooth_positions(positions, alpha=SMOOTHING_ALPHA):
    smoothed = np.empty_like(positions)
    smoothed[0] = positions[0]
    for t in range(1, positions.shape[0]):
        smoothed[t] = alpha * positions[t] + (1 - alpha) * smoothed[t - 1]
    return smoothed.astype(np.float32)
```

def add_dynamics

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
def add_dynamics(positions):
    velocity = np.zeros_like(positions)
    velocity[1:] = positions[1:] - positions[:-1]
    acceleration = np.zeros_like(positions)
    acceleration[1:] = velocity[1:] - velocity[:-1]
    return np.concatenate([positions, velocity, acceleration], axis=1).astype(np.float32)
```

def process_clip

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
def process_clip(landmarker, video_path):
    cap = cv2.VideoCapture(video_path)
    positions, detected, total, last_valid = [], 0, 0, None
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        total += 1
```

```
        landmarks_px = detect_hrnet(landmarker, frame)
        vector = normalize_face_landmarks(landmarks_px) if landmarks_px is not None else None
        if vector is not None and is_outlier(vector, last_valid):
            vector = None
        if vector is not None:
            detected += 1
            last_valid = vector
        positions.append(vector)
    cap.release()
    filled = fill_missing_frames(positions)
    if filled is None:
        return None, detected, total
    return add_dynamics(smooth_positions(filled)), detected, total
```

Imports, constantes y código suelto

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 8

```
FACE_INDICES = list(range(68))
EYE_OUTER_RIGHT = 36
EYE_OUTER_LEFT = 45

POSITION_DIM = len(FACE_INDICES) * 2
LANDMARK_DIM = POSITION_DIM * 3
SCALE_EPSILON = 1e-6
OUTLIER_MAX_JUMP = 0.15
SMOOTHING_ALPHA = 0.4
MIN_DETECTION_RATE_TRAIN = 0.6

print(f'Dimension por frame: {LANDMARK_DIM} = posicion {POSITION_DIM} + velocidad {POSITION_DIM} + aceleracion {POSITION_DIM}')
```

Imports, constantes y código suelto

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 10

```
landmarker = build_hrnet_detector()

label_map = {c: i for i, c in enumerate(clases)}
with open(LABEL_MAP_PATH, 'w', encoding='utf-8') as f:
    json.dump(label_map, f, ensure_ascii=False, indent=2)

total_ok, total_fail = 0, 0
descartados = {}
for word in clases:
    clips = sorted(videos_por_clase[word].items())
    if MAX_CLIPS_POR_CLASE:
        clips = clips[:MAX_CLIPS_POR_CLASE]
    if not clips:
        continue
    out_dir = os.path.join(LANDMARKS_OUT_DIR, word)
    os.makedirs(out_dir, exist_ok=True)
    pendientes = [(n, p) for n, p in clips if not os.path.exists(os.path.join(out_dir, n + '.npy'))]
    print(f'\n[{word}] {len(clips) - len(pendientes)} ya tenian landmarks, {len(pendientes)} por procesar.')
    for i, (nombre, ruta) in enumerate(pendientes, 1):
        sequence, detected, total = process_clip(landmarker, ruta)
        if sequence is None:
            print(f'  [{word} {i}/{len(pendientes)}] {nombre}: NINGUN frame detecto cara -- descartado.')
            total_fail += 1
            continue
        if total and detected / total < MIN_FRACCION_CON_CARA:
            descartados.setdefault(word, []).append(nombre)
            continue
        np.save(os.path.join(out_dir, nombre + '.npy'), sequence)
        total_ok += 1
        if i % 25 == 0 or i == len(pendientes):
            print(f'  [{word} {i}/{len(pendientes)}] {nombre}: cara en {detected}/{total} frames -> {sequence.shape}')

print(f'\n=== TOTAL: {total_ok} guardados | {total_fail} descartados (sin cara en ningun frame) ===')
print('Landmarks de cara completa en:', LANDMARKS_OUT_DIR)
n_baja = sum(len(v) for v in descartados.values())
print(f'Ademas {n_baja} clips se descartaron: la cara completa se veia en menos del {MIN_FRACCION_CON_CARA*100:.0f}% de sus frames
(camara que corta la cara / no se ven los ojos).')
for c, lst in sorted(descartados.items(), key=lambda kv: -len(kv[1])):
    print(f'   {c}: {len(lst)}')
with open(os.path.join(LANDMARKS_OUT_DIR, 'descartados_sin_cara_completa.json'), 'w', encoding='utf-8') as f:
    json.dump(descartados, f, ensure_ascii=False, indent=1)
```

Imports, constantes y código suelto

Colab/Extraer_Cara_Completa_FonoKids.ipynb · celda 12

```
import matplotlib.pyplot as plt
```

```
archivos = sorted(glob.glob(os.path.join(LANDMARKS_OUT_DIR, '*', '*.npy')))
print(f'{len(archivos)} clips con landmarks de cara completa')
if archivos:
    secuencia = np.load(archivos[len(archivos) // 2])
    print('Archivo:', os.path.relpath(archivos[len(archivos) // 2], LANDMARKS_OUT_DIR), '| forma:', secuencia.shape)
    assert secuencia.shape[1] == LANDMARK_DIM, f'dimension inesperada {secuencia.shape[1]}'
    pts = secuencia[secuencia.shape[0] // 2, :POSITION_DIM].reshape(68, 2)
    grupos = {'contorno / mandibula': range(0, 17), 'cejas': range(17, 27), 'nariz': range(27, 36), 'ojos': range(36, 48), 'boca':
range(48, 68)}
    plt.figure(figsize=(5, 6))
    for nombre, idx in grupos.items():
        idx = list(idx)
        plt.scatter(pts[idx, 0], -pts[idx, 1], s=18, label=nombre)
    plt.legend(fontsize=8); plt.axis('equal'); plt.title('Cara normalizada (frame del medio)')
    plt.show()
```

2. Colab · Entrenar el modelo de video (cara completa)

def instalar_si_falta

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 2

```
def instalar_si_falta(modulo, paquete):
    try:
        __import__(modulo)
    except ImportError:
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', paquete])
```

Imports, constantes y código suelto

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 2

```
import os, re, sys, json, math, glob, subprocess

try:
    import google.colab
    EN_COLAB = True
except ImportError:
    EN_COLAB = False
print('Corriendo en:', 'Google Colab' if EN_COLAB else 'tu PC (local)')

if EN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')

import torch
print('CUDA disponible:', torch.cuda.is_available(), '(en Colab: Entorno de ejecucion > Cambiar tipo > GPU)')

import numpy as np
import unicodedata
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader
```

def buscar_raiz_local

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 4

```
def buscar_raiz_local():
    actual = os.path.abspath(os.getcwd())
    for _ in range(5):
        if os.path.isdir(os.path.join(actual, 'data')) and os.path.isdir(os.path.join(actual, 'scripts')):
            return actual
        actual = os.path.dirname(actual)
    return r'D:\FonoKids'
```

Imports, constantes y código suelto

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 4

```
RUTA_VIDEOS = '/content/drive/MyDrive/Aplicaciones_Inteligentes/SACA LANDMARKS/datos/sesiones_continuas'

if not os.path.isdir(RUTA_VIDEOS):
    RUTA_VIDEOS = os.path.join(buscar_raiz_local(), 'data', 'sesiones_continuas')
RUTA_VIDEOS = RUTA_VIDEOS.rstrip('/\\')
CARPETA_SALIDA = os.path.dirname(RUTA_VIDEOS)
assert os.path.isdir(RUTA_VIDEOS), f'No encontre {RUTA_VIDEOS}: cambia RUTA_VIDEOS por la carpeta de tu Drive que tiene los videos'
print('Videos en:     ', RUTA_VIDEOS)
print('Resultados en: ', CARPETA_SALIDA)

LANDMARKS_DIR = os.path.join(CARPETA_SALIDA, 'landmarks_cara_completa_npy')
MODELS_DIR = os.path.join(CARPETA_SALIDA, 'modelos_cara_completa')
OUTPUTS_DIR = os.path.join(CARPETA_SALIDA, 'resultados_cara_completa')
os.makedirs(MODELS_DIR, exist_ok=True)
os.makedirs(OUTPUTS_DIR, exist_ok=True)

assert os.path.isdir(LANDMARKS_DIR), f'No encontre {LANDMARKS_DIR}: corre primero Extraer_Cara_Completa_FonoKids.ipynb'
print('Landmarks de cara completa:', LANDMARKS_DIR)
_conteo = {c: len(glob.glob(os.path.join(LANDMARKS_DIR, c, '*.npy'))) for c in sorted(os.listdir(LANDMARKS_DIR)) if
os.path.isdir(os.path.join(LANDMARKS_DIR, c))}
print(f'{len(_conteo)} clases, {sum(_conteo.values())} clips.')
_pocas = {c: n for c, n in _conteo.items() if n < 20}
if _pocas:
    print('OJO: clases con menos de 20 clips (el modelo las aprende mal):', _pocas)
print('El modelo se guardara en:', MODELS_DIR)
```

def _nfc

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 6

```
def _nfc(text):
    return unicodedata.normalize('NFC', text)
```

class LipReadingDataset

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 6

```
class LipReadingDataset(Dataset):
    def __init__(self, data_dir, max_frames=60, file_paths=None, augment=False):
        self.data_dir = data_dir
        self.max_frames = max_frames
        self.augment = augment

        label_map_path = os.path.join(data_dir, 'label_map.json')
        if not os.path.exists(label_map_path):
            raise FileNotFoundError(f'No encontre {label_map_path}. Corre el notebook de extraccion primero.')
        with open(label_map_path, 'r', encoding='utf-8') as f:
            raw_label_map = json.load(f)
        self.label_map = {_nfc(word): idx for word, idx in raw_label_map.items()}

        num_classes = max(self.label_map.values()) + 1
        self.class_names = ['?'] * num_classes
        for word, idx in self.label_map.items():
            self.class_names[idx] = word

        if file_paths is not None:
            self.file_paths = list(file_paths)
        else:
            self.file_paths = sorted(glob.glob(os.path.join(data_dir, '*', '*.npy')))
        if not self.file_paths:
            raise RuntimeError(f'No hay archivos .npy en {data_dir}.')

        first_sample = np.load(self.file_paths[0])
        self.landmark_dim = int(first_sample.shape[-1])

    def __len__(self):
        return len(self.file_paths)

    def _label_from_path(self, path):
        word = _nfc(os.path.basename(os.path.dirname(path)))
        return self.label_map[word]

    def _apply_augmentation(self, sequence):
        sequence = sequence + np.random.normal(0, 0.003, sequence.shape).astype(np.float32)
        shift = np.random.uniform(-0.01, 0.01, size=(1, sequence.shape[1])).astype(np.float32)
        scale = np.random.uniform(0.97, 1.03)
        sequence = sequence * scale + shift
        T = sequence.shape[0]
        n_mask = max(1, int(round(T * np.random.uniform(0.08, 0.12))))
        n_mask = min(n_mask, T)
        mask_idx = np.random.choice(T, size=n_mask, replace=False)
        sequence[mask_idx] = 0.0
        return sequence.astype(np.float32)

    def __getitem__(self, idx):
        path = self.file_paths[idx]
        sequence = np.load(path).astype(np.float32)
        label = self._label_from_path(path)
        if self.augment:
            sequence = self._apply_augmentation(sequence)
        T = sequence.shape[0]
        if T < self.max_frames:
            pad = np.zeros((self.max_frames - T, sequence.shape[1]), dtype=np.float32)
            sequence = np.concatenate([sequence, pad], axis=0)
            real_length = T
        elif T > self.max_frames:
            sequence = sequence[: self.max_frames]
            real_length = self.max_frames
        else:
            real_length = T
        return (
            torch.from_numpy(sequence),
            torch.tensor(label, dtype=torch.long),
            torch.tensor(real_length, dtype=torch.long),
        )
```

def build_dataloaders

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 6

```
def build_dataloaders(data_dir, max_frames=60, batch_size=32, val_ratio=0.2, seed=42, augment_train=True):
    reference = LipReadingDataset(data_dir=data_dir, max_frames=max_frames)
    file_paths = reference.file_paths

    labels_by_path = {p: reference.label_map[_nfc(os.path.basename(os.path.dirname(p)))] for p in file_paths}
    paths_by_class = {}
    for p in file_paths:
        paths_by_class.setdefault(labels_by_path[p], []).append(p)

    rng = np.random.default_rng(seed)
    train_paths, val_paths = [], []
    for label, paths in paths_by_class.items():
        paths = list(paths)
        rng.shuffle(paths)
        n_val = max(1, int(round(len(paths) * val_ratio)))
        val_paths.extend(paths[:n_val])
        train_paths.extend(paths[n_val:])
    rng.shuffle(train_paths)
    rng.shuffle(val_paths)

    train_ds = LipReadingDataset(data_dir=data_dir, max_frames=max_frames, file_paths=train_paths, augment=augment_train)
    val_ds = LipReadingDataset(data_dir=data_dir, max_frames=max_frames, file_paths=val_paths, augment=False)

    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True, num_workers=0, drop_last=False)
    val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False, num_workers=0, drop_last=False)

    return reference, train_loader, val_loader
```

class PositionalEncoding

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=500):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, : x.size(1)]
```

class TCNBlock

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class TCNBlock(nn.Module):
    def __init__(self, channels, kernel_size=3, dilation=1, dropout=0.1):
        super().__init__()
        padding = (kernel_size - 1) * dilation // 2
        self.conv1 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)
        self.bn1 = nn.BatchNorm1d(channels)
        self.conv2 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)
        self.bn2 = nn.BatchNorm1d(channels)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, key_padding_mask=None):
        residual = x
        x = x.transpose(1, 2)
        if key_padding_mask is not None:
            x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)
        x = self.dropout(self.relu(self.bn1(self.conv1(x))))
        x = self.bn2(self.conv2(x))
        x = x.transpose(1, 2)
        return self.relu(residual + self.dropout(x))
```

class TCNFeatureExtractor

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class TCNFeatureExtractor(nn.Module):
    def __init__(self, channels, n_blocks=3, kernel_size=3, dropout=0.1):
        super().__init__()
        self.blocks = nn.ModuleList([
            TCNBlock(channels, kernel_size=kernel_size, dilation=2 ** i, dropout=dropout)
            for i in range(n_blocks)
```

```
        ])

    def forward(self, x, key_padding_mask=None):
        for block in self.blocks:
            x = block(x, key_padding_mask=key_padding_mask)
        return x
```

class ConformerFeedForward

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class ConformerFeedForward(nn.Module):
    def __init__(self, d_model, dim_feedforward, dropout):
        super().__init__()
        self.net = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, dim_feedforward),
            nn.SiLU(),
            nn.Dropout(dropout),
            nn.Linear(dim_feedforward, d_model),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)
```

class ConformerConvModule

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class ConformerConvModule(nn.Module):
    def __init__(self, d_model, kernel_size=15, dropout=0.1):
        super().__init__()
        self.layer_norm = nn.LayerNorm(d_model)
        self.pointwise_conv1 = nn.Conv1d(d_model, 2 * d_model, kernel_size=1)
        self.glu = nn.GLU(dim=1)
        padding = (kernel_size - 1) // 2
        self.depthwise_conv = nn.Conv1d(d_model, d_model, kernel_size=kernel_size, padding=padding, groups=d_model)
        self.batch_norm = nn.BatchNorm1d(d_model)
        self.swish = nn.SiLU()
        self.pointwise_conv2 = nn.Conv1d(d_model, d_model, kernel_size=1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, key_padding_mask=None):
        residual = x
        x = self.layer_norm(x)
        x = x.transpose(1, 2)
        if key_padding_mask is not None:
            x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)
        x = self.pointwise_conv1(x)
        x = self.glu(x)
        x = self.depthwise_conv(x)
        x = self.batch_norm(x)
        x = self.swish(x)
        x = self.pointwise_conv2(x)
        x = self.dropout(x)
        x = x.transpose(1, 2)
        return residual + x
```

class ConformerBlock

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class ConformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, dim_feedforward, conv_kernel_size=15, dropout=0.1):
        super().__init__()
        self.ff1 = ConformerFeedForward(d_model, dim_feedforward, dropout)
        self.self_attn_norm = nn.LayerNorm(d_model)
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
        self.attn_dropout = nn.Dropout(dropout)
        self.conv_module = ConformerConvModule(d_model, conv_kernel_size, dropout)
        self.ff2 = ConformerFeedForward(d_model, dim_feedforward, dropout)
        self.final_norm = nn.LayerNorm(d_model)

    def forward(self, x, src_key_padding_mask=None):
        x = x + 0.5 * self.ff1(x)
        residual = x
        x_norm = self.self_attn_norm(x)
        attn_out, _ = self.self_attn(x_norm, x_norm, x_norm, key_padding_mask=src_key_padding_mask, need_weights=False)
        x = residual + self.attn_dropout(attn_out)
        x = self.conv_module(x, key_padding_mask=src_key_padding_mask)
        x = x + 0.5 * self.ff2(x)
        return self.final_norm(x)
```

class ConformerEncoder

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class ConformerEncoder(nn.Module):
    def __init__(self, d_model, n_layers, n_heads, dim_feedforward, conv_kernel_size=15, dropout=0.1):
        super().__init__()
        self.layers = nn.ModuleList([
            ConformerBlock(d_model, n_heads, dim_feedforward, conv_kernel_size, dropout)
            for _ in range(n_layers)
        ])

    def forward(self, x, src_key_padding_mask=None):
        for layer in self.layers:
            x = layer(x, src_key_padding_mask=src_key_padding_mask)
        return x
```

class LipReadingConformer

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class LipReadingConformer(nn.Module):
    def __init__(self, num_classes, input_dim, hidden_dim=128, n_layers=2, n_heads=8,
                 dim_feedforward=256, conv_kernel_size=15, tcn_blocks=3, tcn_kernel_size=3, dropout=0.3):
        super().__init__()
        self.input_proj = nn.Linear(input_dim, hidden_dim)
        self.tcn = TCNFeatureExtractor(hidden_dim, n_blocks=tcn_blocks, kernel_size=tcn_kernel_size, dropout=dropout)
        self.pos_encoder = PositionalEncoding(hidden_dim)
        self.conformer_encoder = ConformerEncoder(
            d_model=hidden_dim, n_layers=n_layers, n_heads=n_heads,
            dim_feedforward=dim_feedforward, conv_kernel_size=conv_kernel_size, dropout=dropout,
        )
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, 64), nn.ReLU(), nn.Dropout(dropout), nn.Linear(64, num_classes),
        )

    def forward_features(self, x, src_key_padding_mask=None):
        x = self.input_proj(x)
        x = self.tcn(x, key_padding_mask=src_key_padding_mask)
        x = self.pos_encoder(x)
        x = self.conformer_encoder(x, src_key_padding_mask=src_key_padding_mask)
        return masked_mean_pool(x, src_key_padding_mask)

    def forward(self, x, src_key_padding_mask=None):
        pooled = self.forward_features(x, src_key_padding_mask=src_key_padding_mask)
        return self.classifier(pooled)
```

def masked_mean_pool

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
def masked_mean_pool(x, src_key_padding_mask):
    if src_key_padding_mask is None:
        return x.mean(dim=1)
    real_mask = (~src_key_padding_mask).unsqueeze(-1).float()
    summed = (x * real_mask).sum(dim=1)
    counts = real_mask.sum(dim=1).clamp(min=1.0)
    return summed / counts
```

def make_padding_mask

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
def make_padding_mask(lengths, max_len, device):
    idx = torch.arange(max_len, device=device).unsqueeze(0)
    return idx >= lengths.unsqueeze(1)
```

class EarlyStopping

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
class EarlyStopping:
    def __init__(self, patience=8, min_delta=0.001):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.early_stop = False

    def __call__(self, val_loss):
        if self.best_loss is None or val_loss < self.best_loss - self.min_delta:
            self.best_loss = val_loss
            self.counter = 0
        else:
```

```
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
```

def run_epoch

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
def run_epoch(model, loader, criterion, optimizer, device, train):
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    context = torch.enable_grad() if train else torch.no_grad()
    with context:
        for sequences, labels, lengths in loader:
            sequences = sequences.to(device)
            labels = labels.to(device)
            lengths = lengths.to(device)
            mask = make_padding_mask(lengths, sequences.shape[1], device)
            if train:
                optimizer.zero_grad()
            outputs = model(sequences, src_key_padding_mask=mask)
            loss = criterion(outputs, labels)
            if train:
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
                optimizer.step()
            total_loss += loss.item() * sequences.size(0)
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)
    avg_loss = total_loss / total if total > 0 else 0.0
    acc = 100.0 * correct / total if total > 0 else 0.0
    return avg_loss, acc
```

def build_warmup_cosine_scheduler

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
def build_warmup_cosine_scheduler(optimizer, epochs, warmup_epochs=5):
    warmup_epochs = max(1, min(warmup_epochs, epochs - 1)) if epochs > 1 else 1

    def lr_lambda(epoch):
        if epoch < warmup_epochs:
            return (epoch + 1) / warmup_epochs
        progress = (epoch - warmup_epochs) / max(1, epochs - warmup_epochs)
        return 0.5 * (1 + math.cos(math.pi * progress))

    return optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)
```

Imports, constantes y código suelto

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 8

```
import time
```

Imports, constantes y código suelto

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 10

```
VIDEO_MODEL_PATH = os.path.join(MODELS_DIR, 'best_cara_completa.pth')
SEED = 42
torch.manual_seed(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Entrenando en: {device}')

EPOCHS = 50
BATCH_SIZE = 32
MAX_FRAMES = 60
LR = 1e-3
WEIGHT_DECAY = 1e-3
VAL_RATIO = 0.2
# Con 54 clases la pérdida de validación salta mucho mientras la tasa de aprendizaje es alta; con paciencia 15 el entrenamiento se
cortaba en la época 27,
# antes de que el decaimiento coseno lo estabilizara. Se usa la paciencia = nº de épocas (sin parada anticipada) y se conserva el mejor
modelo por pérdida de validación.
EARLY_STOPPING_PATIENCE = EPOCHS
WARMUP_EPOCHS = 5
LABEL_SMOOTHING = 0.1

dataset, train_loader, val_loader = build_dataloaders(
    data_dir=LANDMARKS_DIR, max_frames=MAX_FRAMES, batch_size=BATCH_SIZE,
    val_ratio=VAL_RATIO, seed=SEED, augment_train=True,
)
```

```
num_classes = len(dataset.class_names)
input_dim = dataset.landmark_dim
print(f'Clases ({num_classes}): {dataset.class_names}')
print(f'Dimension de entrada detectada: {input_dim}')
print(f'Total muestras: {len(dataset)} | Train: {len(train_loader.dataset)} | Val: {len(val_loader.dataset)}')

model = LipReadingConformer(num_classes=num_classes, input_dim=input_dim).to(device)

train_labels = [dataset._label_from_path(p) for p in train_loader.dataset.file_paths]
class_counts = np.bincount(train_labels, minlength=num_classes).astype(np.float64)
# Las clases del label_map que todavia no tienen clips quedan con peso 0: si recibieran el peso
# inflado de dividir por 1, el label smoothing gastaria probabilidad en clases que no existen.
presentes = class_counts > 0
class_weights = np.where(presentes, class_counts.sum() / (presentes.sum() * np.where(presentes, class_counts, 1.0)), 0.0)
print(f'Clases con clips: {int(presentes.sum())} de {num_classes} (las demas llevan peso 0)')
class_weights_t = torch.tensor(class_weights, dtype=torch.float32, device=device)

criterion = nn.CrossEntropyLoss(label_smoothing=LABEL_SMOOTHING, weight=class_weights_t)
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
scheduler = build_warmup_cosine_scheduler(optimizer, EPOCHS, WARMUP_EPOCHS)
early_stopping = EarlyStopping(patience=EARLY_STOPPING_PATIENCE, min_delta=0.001)

best_val_loss = float('inf')
history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

print('\nIniciando entrenamiento (Conformer)...')
for epoch in range(EPOCHS):
    train_loss, train_acc = run_epoch(model, train_loader, criterion, optimizer, device, train=True)
    val_loss, val_acc = run_epoch(model, val_loader, criterion, optimizer, device, train=False)
    scheduler.step()

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['train_acc'].append(train_acc)
    history['val_acc'].append(val_acc)

    lr_actual = optimizer.param_groups[0]['lr']
    print(f'Epoca [{epoch+1}/{EPOCHS}] | Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | '
          f'Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}% | LR: {lr_actual:.6f}')

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save({
            'model_state_dict': model.state_dict(),
            'class_names': dataset.class_names,
            'max_frames': MAX_FRAMES,
            'input_dim': input_dim,
            'landmarks': 'cara_completa_68_normalizada_por_ojos',
            'val_loss': val_loss,
            'val_acc': val_acc,
        }, VIDEO_MODEL_PATH)
        print(f'  -> Nuevo mejor modelo guardado (Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%)')

    early_stopping(val_loss)
    if early_stopping.early_stop:
        print(f'  -> Early stopping en la epoca {epoch+1}.')
        break

print(f'\nListo. Mejor Val Loss: {best_val_loss:.4f} | Modelo en: {VIDEO_MODEL_PATH}')
```

Imports, constantes y código suelto

Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 12

```
from sklearn.metrics import confusion_matrix

epochs_range = range(1, len(history['train_loss']) + 1)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
ax1.plot(epochs_range, history['train_loss'], 'b-', label='Train Loss', linewidth=2)
ax1.plot(epochs_range, history['val_loss'], 'r--', label='Validation Loss', linewidth=2)
ax1.set_title('Convergencia de Perdida (cara completa)')
ax1.set_xlabel('Epocas'); ax1.set_ylabel('Perdida'); ax1.legend(); ax1.grid(True)
ax2.plot(epochs_range, history['train_acc'], 'b-', label='Train Accuracy', linewidth=2)
ax2.plot(epochs_range, history['val_acc'], 'g-', label='Validation Accuracy', linewidth=2)
ax2.set_title('Precision del modelo de cara completa')
ax2.set_xlabel('Epocas'); ax2.set_ylabel('Precision (%)'); ax2.legend(); ax2.grid(True)
plt.tight_layout()
curvas_path = os.path.join(OUTPUTS_DIR, 'cara_completa_curvas.png')
plt.savefig(curvas_path)
plt.show()
print(f'-> Grafico guardado: {curvas_path}')

best_checkpoint = torch.load(VIDEO_MODEL_PATH, map_location=device)
```

```
model.load_state_dict(best_checkpoint['model_state_dict'])
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for sequences, labels, lengths in val_loader:
        sequences = sequences.to(device)
        lengths_dev = lengths.to(device)
        mask = make_padding_mask(lengths_dev, sequences.shape[1], device)
        outputs = model(sequences, src_key_padding_mask=mask)
        preds = outputs.argmax(dim=1).cpu()
        all_preds.extend(preds.tolist())
        all_labels.extend(labels.tolist())

total = len(all_labels)
top1_acc = 100.0 * sum(p == l for p, l in zip(all_preds, all_labels)) / total
print(f'Top-1 accuracy (val): {top1_acc:.2f}%')

cm = confusion_matrix(all_labels, all_preds, labels=list(range(len(dataset.class_names))))
plt.figure(figsize=(max(8, len(dataset.class_names) * 0.35), max(6, len(dataset.class_names) * 0.35)))
plt.imshow(cm, cmap='Blues')
plt.xticks(range(len(dataset.class_names)), dataset.class_names, rotation=90, fontsize=8)
plt.yticks(range(len(dataset.class_names)), dataset.class_names, fontsize=8)
plt.xlabel('Predicho'); plt.ylabel('Real')
plt.title(f'Matriz de confusion (cara completa, val) -- top1 {top1_acc:.1f}%')
plt.colorbar()
plt.tight_layout()
cm_path = os.path.join(OUTPUTS_DIR, 'cara_completa_matriz_confusion.png')
plt.savefig(cm_path)
plt.show()
print(f'-> Matriz de confusion guardada: {cm_path}')
```

Imports, constantes y código suelto

`Colab/Entrenar_Cara_Completa_FonoKids.ipynb · celda 13`

```
all_logits = []
model.eval()
with torch.no_grad():
    for sequences, labels, lengths in val_loader:
        sequences = sequences.to(device)
        mask = make_padding_mask(lengths.to(device), sequences.shape[1], device)
        all_logits.append(model(sequences, src_key_padding_mask=mask).cpu())
logits = torch.cat(all_logits)
etiquetas = torch.tensor(all_labels)
top1 = 100.0 * (logits.argmax(1) == etiquetas).float().mean().item()
top3 = 100.0 * (logits.topk(3, dim=1).indices == etiquetas.unsqueeze(1)).any(1).float().mean().item()
es_correcto = lambda idx: dataset.class_names[idx].endswith('_correcto')
pred = logits.argmax(1).tolist()
binaria = 100.0 * sum(es_correcto(p) == es_correcto(l) for p, l in zip(pred, all_labels)) / len(all_labels)

REFERENCIA_LABIOS = {'top1': 67.17, 'top3': 92.46, 'binaria': 75.98}
n_clases = len(dataset.class_names)
if n_clases == 58:
    print(f"{'':28s} {'Solo labios':>12s} {'Cara completa':>14s} {'Diferencia':>11s}")
    for nombre, clave, valor in (('Top-1 (clase exacta, 58)', 'top1', top1), ('Top-3', 'top3', top3), ('Bien o mal (binaria)',
'binaria', binaria)):
        ref = REFERENCIA_LABIOS[clave]
        print(f'{nombre:28s} {ref:11.2f}% {valor:13.2f}% {valor - ref:+10.2f}')
else:
    print(f'Cara completa ({n_clases} clases): Top-1 {top1:.2f}% | Top-3 {top3:.2f}% | Bien o mal {binaria:.2f}%')
    print('Las cifras del modelo de solo labios (67.17 % / 92.46 % / 75.98 %) son de 58 clases con el dataset completo: con otras clases
u otra cantidad de clips NO son comparables.')
    print('Para comparar justo: entrena el modelo de labios con Entrenar_Modelos_FonoKids.ipynb sobre los MISMOS clips (sus landmarks de
boca) y compara con estas cifras.')
print(f'\nClips de validacion: {len(all_labels)} | Entrada: {input_dim} numeros por frame')

with open(os.path.join(OUTPUTS_DIR, 'cara_completa_resultados.json'), 'w', encoding='utf-8') as f:
    json.dump({'top1': top1, 'top3': top3, 'binaria': binaria, 'referencia_labios': REFERENCIA_LABIOS,
               'clips_validacion': len(all_labels), 'input_dim': int(input_dim), 'clases': len(dataset.class_names)}, f, indent=2)
print('Resultados guardados en', os.path.join(OUTPUTS_DIR, 'cara_completa_resultados.json'))
```

3. Audio · fonemas, GOP y embeddings (scripts/audio_pronunciation.py)

def texto_a_fonemas

scripts/audio_pronunciation.py:22-126

```
def texto_a_fonemas(palabra):
    palabra = palabra.strip().lower()
    for acentuada, simple in VOCALES_ACENTUADAS.items():
        palabra = palabra.replace(acentuada, simple)

    fonemas = []
    i = 0
    n = len(palabra)
    while i < n:
        c = palabra[i]
        nxt = palabra[i + 1] if i + 1 < n else ""
        prev = palabra[i - 1] if i > 0 else ""

        if c == "r":
            doble = nxt == "r"
            al_inicio = i == 0
            tras_nls = prev in ("n", "l", "s")
            fonemas.append("r" if (doble or al_inicio or tras_nls) else "ɾ")
            i += 2 if doble else 1
            continue

        if c == "c" and nxt in ("e", "i"):
            fonemas.append("s")
            i += 1
            continue
        if c == "c" and nxt == "h":
            fonemas.append("tʃ")
            i += 2
            continue
        if c == "c":
            fonemas.append("k")
            i += 1
            continue

        if c == "q" and nxt == "u":
            fonemas.append("k")
            i += 2
            continue

        if c == "g" and nxt == "u" and i + 2 < n and palabra[i + 2] in ("e", "i"):
            fonemas.append("g")
            i += 2
            continue
        if c == "g" and nxt in ("e", "i"):
            fonemas.append("x")
            i += 1
            continue
        if c == "g":
            fonemas.append("g")
            i += 1
            continue

        if c == "l" and nxt == "l":
            fonemas.append("ʝ")
            i += 2
            continue

        if c == "y":
            es_vocal_final = (i == n - 1) and prev in "aeiou"
            fonemas.append("i" if es_vocal_final else "ʝ")
            i += 1
            continue

        if c == "z":
            fonemas.append("s")
            i += 1
            continue

        if c == "j":
            fonemas.append("x")
            i += 1
            continue

        if c == "h":
            i += 1
```

```
            continue

        if c == "x":
            fonemas.extend(["k", "s"])
            i += 1
            continue

        if c == "v":
            fonemas.append("b")
            i += 1
            continue

        if c == "ñ":
            fonemas.append("ɲ")
            i += 1
            continue

        if c in "aeiou":
            fonemas.append(c)
            i += 1
            continue

        if c in "ptdkbfmnlsw":
            fonemas.append(c)
            i += 1
            continue

        i += 1

    return fonemas
```

def load_model

scripts/audio_pronunciation.py:129-152

```
def load_model(device=None):
    import json
    from huggingface_hub import hf_hub_download
    from transformers import Wav2Vec2Config, Wav2Vec2FeatureExtractor, Wav2Vec2ForCTC

    device = device or ("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Cargando {MODEL_NAME} (primera vez tarda, baja ~1.2GB)...")

    feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(MODEL_NAME)

    config = Wav2Vec2Config.from_pretrained(MODEL_NAME)
    model = Wav2Vec2ForCTC(config)
    weights_path = hf_hub_download(MODEL_NAME, "pytorch_model.bin")
    state_dict = torch.load(weights_path, map_location="cpu", weights_only=True)
    model.load_state_dict(state_dict)
    model.to(device)
    model.eval()

    vocab_path = hf_hub_download(MODEL_NAME, "vocab.json")
    with open(vocab_path, encoding="utf-8") as f:
        vocab = json.load(f)

    print(f"Listo. Device: {device}. Vocabulario: {len(vocab)} fonemas.")
    return model, feature_extractor, vocab, device
```

def load_audio_16k

scripts/audio_pronunciation.py:155-161

```
def load_audio_16k(wav_path):
    waveform, sr = torchaudio.load(wav_path)
    if waveform.shape[0] > 1:
        waveform = waveform.mean(dim=0, keepdim=True)
    if sr != TARGET_SAMPLE_RATE:
        waveform = torchaudio.functional.resample(waveform, sr, TARGET_SAMPLE_RATE)
    return waveform.squeeze(0)
```

def extract_embedding

scripts/audio_pronunciation.py:167-173

```
def extract_embedding(wav_path, model, feature_extractor, device):
    waveform = load_audio_16k(wav_path)
    inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")
    with torch.inference_mode():
        hidden = model.wav2vec2(inputs.input_values.to(device)).last_hidden_state
        pooled = hidden.mean(dim=1).squeeze(0)
    return pooled.cpu().numpy()
```

def _phoneme_to_token_ids

scripts/audio_pronunciation.py:176-185

```
def _phoneme_to_token_ids(fonemas, vocab):
    ids = []
    fonemas_usados = []
    for f in fonemas:
        if f in vocab:
            ids.append(vocab[f])
            fonemas_usados.append(f)
        else:
            print(f"[AVISO] Fonema '{f}' no está en el vocabulario del modelo, se saltea.")
    return ids, fonemas_usados
```

def _greedy_ctc_decode

scripts/audio_pronunciation.py:188-195

```
def _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id):
    fonemas = []
    anterior = None
    for tid in pred_ids:
        if tid != anterior and tid != blank_id:
            fonemas.append(id_to_phoneme.get(tid, "?"))
        anterior = tid
    return fonemas
```

def fonemas_a_texto_aproximado

scripts/audio_pronunciation.py:201-202

```
def fonemas_a_texto_aproximado(fonemas):
    return "".join(_FONEMA_A_LETRA.get(f, f) for f in fonemas)
```

def _distancia_edicion

scripts/audio_pronunciation.py:205-218

```
def _distancia_edicion(a, b):
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[n][m]
```

def _align_ops

scripts/audio_pronunciation.py:221-253

```
def _align_ops(a, b):
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])

    ops = []
    i, j = n, m
    while i > 0 or j > 0:
        if i > 0 and j > 0 and a[i - 1] == b[j - 1]:
            ops.append(("match", a[i - 1], b[j - 1]))
            i -= 1
            j -= 1
        elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1:
            ops.append(("sub", a[i - 1], b[j - 1]))
            i -= 1
            j -= 1
        elif i > 0 and dp[i][j] == dp[i - 1][j] + 1:
```

```
            ops.append(("del", a[i - 1], None))
            i -= 1
        else:
            ops.append(("ins", None, b[j - 1]))
            j -= 1
    ops.reverse()
    return ops
```

def diagnose_tipo_error

scripts/audio_pronunciation.py:268-293

```
def diagnose_tipo_error(fonemas_esperados, reconocido_libre_lista):
    ops = _align_ops(fonemas_esperados, reconocido_libre_lista)

    for idx, (tipo, esperado, reconocido) in enumerate(ops):
        if esperado == "l" and idx > 0:
            anterior = ops[idx - 1][1]
            if anterior and (anterior + "l") in _SINFONES:
                if tipo == "del":
                    return "omision"
                if tipo == "sub" and reconocido and reconocido not in ("l",):
                    return "distorsion"
    for idx, (tipo, esperado, reconocido) in enumerate(ops):
        if tipo == "ins" and idx > 0:
            previo = ops[idx - 1]
            if previo[0] in ("match", "sub") and previo[1] and (previo[1] + "l") in _SINFONES:
                return "epentesis"

    for tipo, esperado, reconocido in ops:
        tabla = _SUSTITUCIONES_CONOCIDAS.get(esperado)
        if tabla is None:
            continue
        if tipo == "del":
            return "omision"
        if tipo == "sub" and reconocido in tabla:
            return tabla[reconocido]
    return None
```

def score_pronunciation

scripts/audio_pronunciation.py:296-361

```
def score_pronunciation(wav_path, palabra, model, feature_extractor, vocab, device):
    waveform = load_audio_16k(wav_path)

    inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")
    with torch.inference_mode():
        logits = model(inputs.input_values.to(device)).logits
    log_probs = torch.log_softmax(logits, dim=-1)

    id_to_phoneme = {v: k for k, v in vocab.items()}
    blank_id = vocab["<pad>"]

    pred_ids = torch.argmax(log_probs, dim=-1)[0].tolist()
    reconocido_libre = _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id)

    fonemas_esperados = texto_a_fonemas(palabra)
    target_ids, fonemas_usados = _phoneme_to_token_ids(fonemas_esperados, vocab)
    if not target_ids:
        raise ValueError(f"No se pudo mapear ningún fonema de '{palabra}' al vocabulario del modelo.")

    targets = torch.tensor([target_ids], dtype=torch.int32, device=device)

    aligned_labels, aligned_scores = F.forced_align(
        log_probs, targets, blank=blank_id,
    )
    aligned_labels = aligned_labels[0].tolist()
    aligned_scores = aligned_scores[0].tolist()

    scores_por_fonema = []
    idx_target = 0
    frame_scores_actual = []
    for label, score in zip(aligned_labels, aligned_scores):
        if label == blank_id:
            continue
        if idx_target < len(target_ids) and label == target_ids[idx_target]:
            frame_scores_actual.append(score)
        elif frame_scores_actual:
            scores_por_fonema.append(float(np.mean(frame_scores_actual)))
            frame_scores_actual = [score]
            idx_target += 1
        else:
```

```
            frame_scores_actual = [score]
    if frame_scores_actual:
        scores_por_fonema.append(float(np.mean(frame_scores_actual)))

    while len(scores_por_fonema) < len(fonemas_usados):
        scores_por_fonema.append(None)

    sospechosos = [
        i for i, s in enumerate(scores_por_fonema)
        if s is not None and s < GOP_SUSPICIOUS_THRESHOLD
    ]

    distancia = _distancia_edicion(fonemas_usados, reconocido_libre)
    normalizada = distancia / max(len(fonemas_usados), len(reconocido_libre), 1)
    palabra_reconocida = normalizada <= PALABRA_MISMATCH_THRESHOLD

    return {
        "fonemas": fonemas_usados,
        "scores": scores_por_fonema,
        "sospechosos": sospechosos,
        "reconocido_libre": " ".join(reconocido_libre),
        "sonido_reconocido": fonemas_a_texto_aproximado(reconocido_libre),
        "distancia_normalizada": normalizada,
        "palabra_reconocida": palabra_reconocida,
        "tipo_error_audio": diagnose_tipo_error(fonemas_usados, reconocido_libre),
    }
```

def _localizar_subsecuencia

scripts/audio_pronunciation.py:369-382

```
def _localizar_subsecuencia(secuencia, objetivo):
    n, m = len(secuencia), len(objetivo)
    if m == 0 or n == 0:
        return 0, 0
    mejor_inicio, mejor_fin, mejor_dist = 0, min(m, n), None
    for largo in (max(1, m - 1), m, m + 1):
        for inicio in range(0, max(1, n - largo + 1)):
            fin = inicio + largo
            if fin > n:
                continue
            dist = _distancia_edicion(secuencia[inicio:fin], objetivo)
            if mejor_dist is None or dist < mejor_dist:
                mejor_inicio, mejor_fin, mejor_dist = inicio, fin, dist
    return mejor_inicio, mejor_fin
```

def score_word_in_sentence

scripts/audio_pronunciation.py:385-498

```
def score_word_in_sentence(wav_path, frase, palabra_objetivo, model, feature_extractor, vocab, device):
    waveform = load_audio_16k(wav_path)

    inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")
    with torch.inference_mode():
        logits = model(inputs.input_values.to(device)).logits
    log_probs = torch.log_softmax(logits, dim=-1)

    id_to_phoneme = {v: k for k, v in vocab.items()}
    blank_id = vocab["<pad>"]

    pred_ids = torch.argmax(log_probs, dim=-1)[0].tolist()
    reconocido_libre = _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id)

    fonemas_frase = texto_a_fonemas(frase)
    fonemas_palabra = texto_a_fonemas(palabra_objetivo)
    target_ids, fonemas_frase_usados = _phoneme_to_token_ids(fonemas_frase, vocab)
    if not target_ids:
        raise ValueError(f"No se pudo mapear ningún fonema de la frase '{frase}' al vocabulario del modelo.")

    targets = torch.tensor([target_ids], dtype=torch.int32, device=device)
    aligned_labels, aligned_scores = F.forced_align(log_probs, targets, blank=blank_id)
    aligned_labels = aligned_labels[0].tolist()
    aligned_scores = aligned_scores[0].tolist()

    scores_por_fonema = []
    frame_ranges = []
    idx_target = 0
    frame_scores_actual = []
    frame_indices_actual = []
    for frame_idx, (label, score) in enumerate(zip(aligned_labels, aligned_scores)):
        if label == blank_id:
            continue
```

```
        if idx_target < len(target_ids) and label == target_ids[idx_target]:
            frame_scores_actual.append(score)
            frame_indices_actual.append(frame_idx)
        elif frame_scores_actual:
            scores_por_fonema.append(float(np.mean(frame_scores_actual)))
            frame_ranges.append((frame_indices_actual[0], frame_indices_actual[-1]))
            frame_scores_actual = [score]
            frame_indices_actual = [frame_idx]
            idx_target += 1
        else:
            frame_scores_actual = [score]
            frame_indices_actual = [frame_idx]
    if frame_scores_actual:
        scores_por_fonema.append(float(np.mean(frame_scores_actual)))
        frame_ranges.append((frame_indices_actual[0], frame_indices_actual[-1]))
    while len(scores_por_fonema) < len(fonemas_frase_usados):
        scores_por_fonema.append(None)
        frame_ranges.append(None)

    inicio, fin = _localizar_subsecuencia(fonemas_frase_usados, fonemas_palabra)
    fonemas_usados = fonemas_frase_usados[inicio:fin]
    scores_recortados = scores_por_fonema[inicio:fin]

    rangos_validos = [r for r in frame_ranges[inicio:fin] if r is not None]
    audio_recortado = None
    if rangos_validos:
        total_frames = log_probs.shape[1]
        total_muestras = waveform.shape[0]
        muestras_por_frame = total_muestras / max(total_frames, 1)
        frame_inicio = min(r[0] for r in rangos_validos)
        frame_fin = max(r[1] for r in rangos_validos)
        colchon_muestras = int(0.15 * TARGET_SAMPLE_RATE)
        muestra_inicio = max(0, int(frame_inicio * muestras_por_frame) - colchon_muestras)
        muestra_fin = min(total_muestras, int((frame_fin + 1) * muestras_por_frame) + colchon_muestras)
        if muestra_fin > muestra_inicio:
            audio_recortado = waveform[muestra_inicio:muestra_fin]

    sospechosos = [
        i for i, s in enumerate(scores_recortados)
        if s is not None and s < GOP_SUSPICIOUS_THRESHOLD
    ]

    scores_fuera_de_la_palabra = scores_por_fonema[:inicio] + scores_por_fonema[fin:]
    sospechosos_fuera = [
        s for s in scores_fuera_de_la_palabra
        if s is not None and s < GOP_SUSPICIOUS_THRESHOLD
    ]
    resto_de_la_frase_ok = len(sospechosos_fuera) < 2

    lib_inicio, lib_fin = _localizar_subsecuencia(reconocido_libre, fonemas_palabra)
    ventana_reconocida = reconocido_libre[lib_inicio:lib_fin]
    distancia = _distancia_edicion(ventana_reconocida, fonemas_palabra)
    normalizada = distancia / max(len(fonemas_palabra), len(ventana_reconocida), 1)
    palabra_reconocida = normalizada <= FRASE_PALABRA_MISMATCH_THRESHOLD

    frase_completa_dicha = (
        len(reconocido_libre) >= FRASE_COMPLETA_LARGO_MINIMO * len(fonemas_frase_usados)
    )

    audio_recortado_path = None
    if audio_recortado is not None:
        import tempfile
        with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp:
            audio_recortado_path = tmp.name
        torchaudio.save(audio_recortado_path, audio_recortado.unsqueeze(0), TARGET_SAMPLE_RATE)

    return {
        "frase": frase,
        "palabra_objetivo": palabra_objetivo,
        "fonemas": fonemas_usados,
        "scores": scores_recortados,
        "sospechosos": sospechosos,
        "reconocido_libre": " ".join(reconocido_libre),
        "sonido_reconocido": fonemas_a_texto_aproximado(ventana_reconocida),
        "distancia_normalizada": normalizada,
        "palabra_reconocida": palabra_reconocida,
        "frase_completa_dicha": frase_completa_dicha,
        "resto_de_la_frase_ok": resto_de_la_frase_ok,
        "tipo_error_audio": diagnose_tipo_error(fonemas_palabra, reconocido_libre),
        "audio_recortado_path": audio_recortado_path,
    }
```

def print_report

scripts/audio_pronunciation.py:501-518

```
def print_report(palabra, resultado):
    print(f"\nPalabra objetivo: '{palabra}'")
    print(f"Fonemas esperados: {' '.join(resultado['fonemas'])}")
    print(f"Lo que reconoció el modelo (libre, sin forzar): {resultado['reconocido_libre']}")
    print("Score por fonema (más alto = más claro, más bajo = más dudoso):")
    for i, (fon, score) in enumerate(zip(resultado["fonemas"], resultado["scores"])):
        marca = " <-- sospechoso" if i in resultado["sospechosos"] else ""
        score_str = f"{score:.2f}" if score is not None else "N/A"
        print(f"  {fon:>4s}: {score_str}{marca}")
    if resultado["sospechosos"]:
        letras = [resultado["fonemas"][i] for i in resultado["sospechosos"]]
        print(f"\n-> Posible error de pronunciación en: {', '.join(letras)}")
    else:
        print("\n-> Todos los fonemas dentro de lo esperado.")
    if not resultado["palabra_reconocida"]:
        print(f"-> OJO: lo reconocido libremente se parece poco a '{palabra}' "
              f"(distancia normalizada {resultado['distancia_normalizada']:.2f}) -- "
              "probablemente dijo otra palabra.")
```

Imports, constantes y código suelto

scripts/audio_pronunciation.py:1-529

```
import sys

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

import numpy as np
import torch
import torchaudio
import torchaudio.functional as F

MODEL_NAME = "facebook/wav2vec2-xlsr-53-espeak-cv-ft"
TARGET_SAMPLE_RATE = 16000

GOP_SUSPICIOUS_THRESHOLD = -3.0

PALABRA_MISMATCH_THRESHOLD = 0.5


VOCALES_ACENTUADAS = {"á": "a", "é": "e", "í": "i", "ó": "o", "ú": "u", "ü": "u"}

EMBEDDING_DIM = 1024

_FONEMA_A_LETRA = {"r": "rr", "ɾ": "r", "x": "j", "g": "g", "tʃ": "ch", "ʝ": "ll", "ɲ": "ñ"}

_SUSTITUCIONES_CONOCIDAS = {
    "r": {"l": "lambdacismo", "d": "dentalizacion"},
    "ɾ": {"l": "lambdacismo", "d": "dentalizacion"},
    "k": {"t": "anteriorizacion", "x": "debilitamiento", "h": "debilitamiento"},
    "g": {"d": "anteriorizacion", "x": "debilitamiento", "h": "debilitamiento"},
    "s": {"θ": "interdental", "t": "oclusion", "d": "oclusion"},
    "ʝ": {"l": "distorsion", "i": "debilitamiento"},
    "tʃ": {"ʃ": "distorsion", "t": "debilitamiento", "s": "debilitamiento"},
}
_SINFONES = {"bl", "kl", "fl", "gl", "pl"}

FRASE_PALABRA_MISMATCH_THRESHOLD = 0.5

FRASE_COMPLETA_LARGO_MINIMO = 0.6

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Uso: python audio_pronunciation.py <ruta_al_wav> <palabra>")
        sys.exit(1)

    wav_path, palabra = sys.argv[1], sys.argv[2]
    model, feature_extractor, vocab, device = load_model()
    resultado = score_pronunciation(wav_path, palabra, model, feature_extractor, vocab, device)
    print_report(palabra, resultado)
```

4. Audio · entrenamiento del clasificador (scripts/train_audio_classifier.py)

def find_labeled_clips

scripts/train_audio_classifier.py:29-52

```
def find_labeled_clips():
    if os.path.isdir(CACHE_DIR):
        por_clase = {}
        for f in sorted(os.listdir(CACHE_DIR)):
            if not f.endswith(".npy"):
                continue
            clase = re.sub(r"_\d+$", "", os.path.splitext(f)[0])
            por_clase.setdefault(clase, []).append(os.path.join(CACHE_DIR, f))
        if por_clase:
            return por_clase

    clips_por_clase = {}
    if os.path.isdir(SESSIONS_DIR):
        for clase in sorted(os.listdir(SESSIONS_DIR)):
            clips_dir = os.path.join(SESSIONS_DIR, clase, "clips")
            if not os.path.isdir(clips_dir):
                continue
            wavs = sorted(
                os.path.join(clips_dir, f) for f in os.listdir(clips_dir) if f.lower().endswith(".wav")
            )
            if wavs:
                clips_por_clase[clase] = wavs

    return clips_por_clase
```

def build_embeddings

scripts/train_audio_classifier.py:55-78

```
def build_embeddings(clips_por_clase, model, feature_extractor, device):
    os.makedirs(CACHE_DIR, exist_ok=True)
    class_names = sorted(clips_por_clase.keys())
    X, y = [], []
    total = sum(len(v) for v in clips_por_clase.values())
    done = 0
    for clase in class_names:
        for item_path in clips_por_clase[clase]:
            if item_path.lower().endswith(".npy"):
                emb = np.load(item_path)
            else:
                cache_name = os.path.splitext(os.path.basename(item_path))[0] + ".npy"
                cache_path = os.path.join(CACHE_DIR, cache_name)
                if os.path.exists(cache_path):
                    emb = np.load(cache_path)
                else:
                    emb = extract_embedding(item_path, model, feature_extractor, device)
                    np.save(cache_path, emb)
            X.append(emb)
            y.append(class_names.index(clase))
            done += 1
            if done % 20 == 0 or done == total:
                print(f"  embeddings: {done}/{total}")
    return np.stack(X).astype(np.float32), np.array(y, dtype=np.int64), class_names
```

def stratified_split

scripts/train_audio_classifier.py:81-92

```
def stratified_split(y, val_fraction, seed):
    rng = np.random.RandomState(seed)
    train_idx, val_idx = [], []
    for c in np.unique(y):
        idx = np.where(y == c)[0]
        rng.shuffle(idx)
        n_val = max(1, int(round(len(idx) * val_fraction)))
        val_idx.extend(idx[:n_val])
        train_idx.extend(idx[n_val:])
    rng.shuffle(train_idx)
    rng.shuffle(val_idx)
    return np.array(train_idx), np.array(val_idx)
```

class AudioClassifierHead

scripts/train_audio_classifier.py:95-107

```
class AudioClassifierHead(nn.Module):

    def __init__(self, in_dim, hidden_dim, num_classes, dropout):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, num_classes),
        )

    def forward(self, x):
        return self.net(x)
```

class EarlyStopping

scripts/train_audio_classifier.py:110-126

```
class EarlyStopping:

    def __init__(self, patience=15, min_delta=0.001):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.early_stop = False

    def __call__(self, val_loss):
        if self.best_loss is None or val_loss < self.best_loss - self.min_delta:
            self.best_loss = val_loss
            self.counter = 0
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
```

def plot_training_curves

scripts/train_audio_classifier.py:129-148

```
def plot_training_curves(history):
    epochs = range(1, len(history["train_loss"]) + 1)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    ax1.plot(epochs, history["train_loss"], "b-", label="Train Loss", linewidth=2)
    ax1.plot(epochs, history["val_loss"], "r--", label="Validation Loss", linewidth=2)
    ax1.set_title("Perdida (audio)")
    ax1.set_xlabel("Epocas"); ax1.set_ylabel("Perdida"); ax1.legend(); ax1.grid(True)

    ax2.plot(epochs, history["train_acc"], "b-", label="Train Accuracy", linewidth=2)
    ax2.plot(epochs, history["val_acc"], "g-", label="Validation Accuracy", linewidth=2)
    ax2.set_title("Precision del clasificador de audio")
    ax2.set_xlabel("Epocas"); ax2.set_ylabel("Precision (%)"); ax2.legend(); ax2.grid(True)

    plt.tight_layout()
    out_path = os.path.join(OUTPUTS_DIR, "audio_classifier_curvas.png")
    os.makedirs(OUTPUTS_DIR, exist_ok=True)
    plt.savefig(out_path)
    plt.close()
    print(f"-> Grafico guardado: {out_path}")
```

def plot_confusion_matrix

scripts/train_audio_classifier.py:151-165

```
def plot_confusion_matrix(conf, class_names, val_acc):
    n = len(class_names)
    fig, ax = plt.subplots(figsize=(max(6, n * 0.45), max(5, n * 0.45)))
    im = ax.imshow(conf, cmap="Blues")
    ax.set_xticks(range(n)); ax.set_xticklabels(class_names, rotation=90, fontsize=8)
    ax.set_yticks(range(n)); ax.set_yticklabels(class_names, fontsize=8)
    ax.set_xlabel("Predicho"); ax.set_ylabel("Real")
    ax.set_title(f"Matriz de confusion (audio) -- val acc {val_acc*100:.1f}%")
    plt.colorbar(im)
    plt.tight_layout()
    out_path = os.path.join(OUTPUTS_DIR, "audio_matriz_confusion.png")
    os.makedirs(OUTPUTS_DIR, exist_ok=True)
    plt.savefig(out_path)
```

```
    plt.close()
    print(f"-> Matriz de confusion guardada: {out_path}")
```

def main

scripts/train_audio_classifier.py:168-284

```
def main():
    clips_por_clase = find_labeled_clips()
    if not clips_por_clase:
        print(f"No encontré clips con audio en {SESSIONS_DIR}. Grabá primero con "
              "grabar_video_continuo.py.")
        return
    print("Clases encontradas:")
    for clase, wavs in clips_por_clase.items():
        print(f"  {clase}: {len(wavs)} clips")

    device = "cuda" if torch.cuda.is_available() else "cpu"
    todos_cacheados = all(
        item.lower().endswith(".npy") for wavs in clips_por_clase.values() for item in wavs
    )
    if todos_cacheados:
        print("\nTodos los embeddings ya están cacheados -- no hace falta cargar wav2vec2.")
        model = feature_extractor = None
    else:
        print(f"\nCargando wav2vec2 para extraer embeddings (device={device})...")
        model, feature_extractor, _vocab, device = load_model(device=device)

    print("\nExtrayendo embeddings (con caché, las corridas siguientes son instantáneas)...")
    X, y, class_names = build_embeddings(clips_por_clase, model, feature_extractor, device)
    print(f"\nTotal: {len(X)} clips, {len(class_names)} clases: {class_names}")

    train_idx, val_idx = stratified_split(y, VAL_FRACTION, SEED)
    print(f"Train: {len(train_idx)} | Val: {len(val_idx)}")

    mean = X[train_idx].mean(axis=0, keepdims=True)
    std = X[train_idx].std(axis=0, keepdims=True) + 1e-6

    def to_tensor(idx):
        xs = (X[idx] - mean) / std
        return torch.from_numpy(xs).float().to(device), torch.from_numpy(y[idx]).long().to(device)

    x_train, y_train = to_tensor(train_idx)
    x_val, y_val = to_tensor(val_idx)

    head = AudioClassifierHead(EMBEDDING_DIM, HIDDEN_DIM, len(class_names), DROPOUT).to(device)
    optimizer = torch.optim.Adam(head.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
    criterion = nn.CrossEntropyLoss()

    history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
    best_val_acc = -1.0
    best_state = None
    early_stopping = EarlyStopping(patience=EARLY_STOPPING_PATIENCE, min_delta=0.001)
    for epoch in range(1, EPOCHS + 1):
        head.train()
        optimizer.zero_grad()
        logits = head(x_train)
        loss = criterion(logits, y_train)
        loss.backward()
        optimizer.step()

        head.eval()
        with torch.no_grad():
            train_acc = (head(x_train).argmax(1) == y_train).float().mean().item()
            val_logits = head(x_val)
            val_loss = criterion(val_logits, y_val).item()
            val_acc = (val_logits.argmax(1) == y_val).float().mean().item()

        history["train_loss"].append(loss.item())
        history["val_loss"].append(val_loss)
        history["train_acc"].append(train_acc * 100)
        history["val_acc"].append(val_acc * 100)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_state = {k: v.clone() for k, v in head.state_dict().items()}

        if epoch % 10 == 0 or epoch == 1:
            print(f"  epoch {epoch:3d} | loss {loss.item():.3f} | val_loss {val_loss:.3f} | "
                  f"train_acc {train_acc:.2f} | val_acc {val_acc:.2f}")

        early_stopping(val_loss)
        if early_stopping.early_stop:
```

```
            print(f"  -> Early stopping activado en la epoca {epoch} "
                  f"(sin mejora en val_loss por {EARLY_STOPPING_PATIENCE} epocas).")
            break

    plot_training_curves(history)

    print(f"\nMejor accuracy de validación: {best_val_acc:.2f} ({int(best_val_acc*len(val_idx))}/{len(val_idx)})")
    if len(val_idx) < 10:
        print("[AVISO] Set de validación muy chico (<10 clips) -- este número es orientativo, "
              "no una medición confiable todavía. Va a mejorar con más datos.")

    head.load_state_dict(best_state)
    head.eval()
    with torch.no_grad():
        val_pred = head(x_val).argmax(1).cpu().numpy()
    val_true = y[val_idx]
    print("\nMatriz de confusión (filas=real, columnas=predicho):")
    print("clases:", class_names)
    n = len(class_names)
    conf = np.zeros((n, n), dtype=int)
    for t, p in zip(val_true, val_pred):
        conf[t, p] += 1
    for i, row in enumerate(conf):
        print(f"  {class_names[i]:35s} {row.tolist()}")

    plot_confusion_matrix(conf, class_names, best_val_acc)

    os.makedirs(os.path.dirname(MODEL_OUT), exist_ok=True)
    torch.save(
        {
            "state_dict": best_state,
            "class_names": class_names,
            "embedding_dim": EMBEDDING_DIM,
            "hidden_dim": HIDDEN_DIM,
            "mean": mean,
            "std": std,
            "val_acc": best_val_acc,
        },
        MODEL_OUT,
    )
    print(f"\nGuardado en {MODEL_OUT}")
```

Imports, constantes y código suelto

`scripts/train_audio_classifier.py:1-288`

```
import os
import re
import sys

import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from audio_pronunciation import EMBEDDING_DIM, extract_embedding, load_model

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
SESSIONS_DIR = os.path.join(BASE_DIR, "data", "sesiones_continuas")
CACHE_DIR = os.path.join(BASE_DIR, "data", "audio_embeddings_cache")
MODEL_OUT = os.path.join(BASE_DIR, "models", "audio_classifier.pth")
OUTPUTS_DIR = os.path.join(BASE_DIR, "outputs")

VAL_FRACTION = 0.2
SEED = 42
EPOCHS = 150
HIDDEN_DIM = 128
DROPOUT = 0.4
WEIGHT_DECAY = 1e-3
LR = 1e-3
EARLY_STOPPING_PATIENCE = 15

if __name__ == "__main__":
    main()
```

5. Servidor FastAPI (server/main.py)

def _parse_clase_predicha

server/main.py:75-86

```
def _parse_clase_predicha(nombre_clase):
    if nombre_clase.endswith(SUFIJO_CORRECTO):
        return nombre_clase[: -len(SUFIJO_CORRECTO)], True, None

    idx = nombre_clase.find(MARCADOR_INCORRECTO)
    if idx != -1:
        palabra = nombre_clase[:idx]
        resto = nombre_clase[idx + len(MARCADOR_INCORRECTO):]
        subtipo = resto[1:] if resto.startswith("_") else None
        return palabra, False, subtipo or None

    return nombre_clase, None, None
```

def build_hrnet_detector

server/main.py:97-110

```
def build_hrnet_detector():
    import face_alignment
    import torch._dynamo

    torch._dynamo.config.suppress_errors = True

    if DEVICE == "cpu":
        print("[AVISO] No hay GPU disponible -- HRNet/FAN en CPU va a ser MUY lento.")

    detector = face_alignment.FaceAlignment(
        face_alignment.LandmarksType.TWO_D, device=DEVICE, flip_input=False, compile=False,
    )
    print(f">>> Detector HRNet/FAN corriendo en {DEVICE.upper()}. <<<")
    return detector
```

def detect_landmarks_batch

server/main.py:113-154

```
def detect_landmarks_batch(detector, frames_bgr):
    if not frames_bgr:
        return []

    frames_rgb = [cv2.cvtColor(f, cv2.COLOR_BGR2RGB) for f in frames_bgr]
    device = detector.face_detector.device
    chunk_size = HRNET_BATCH_CHUNK_SIZE
    detected_faces = []
    try:
        for start in range(0, len(frames_rgb), chunk_size):
            chunk = frames_rgb[start:start + chunk_size]
            batch = torch.stack([
                torch.from_numpy(np.ascontiguousarray(f)).permute(2, 0, 1) for f in chunk
            ]).to(device)
            try:
                chunk_faces = detector.face_detector.detect_from_batch(batch)
            except torch.cuda.OutOfMemoryError:
                del batch
                print(f"[AVISO] Sub-lote de {len(chunk)} frames sin memoria, reintentando de a un frame.")
                chunk_faces = []
                for frame in chunk:
                    single = torch.from_numpy(np.ascontiguousarray(frame)).permute(2, 0, 1).unsqueeze(0).to(device)
                    chunk_faces.extend(detector.face_detector.detect_from_batch(single))
                    del single
            detected_faces.extend(chunk_faces)
            del batch
    except Exception as e:
        print(f"[AVISO] Deteccion en batch fallo ({e}), usando frame por frame.")
        detected_faces = None

    resultados = []
    for i, frame_rgb in enumerate(frames_rgb):
        if detected_faces is None:
            preds = detector.get_landmarks_from_image(frame_rgb)
        else:
            faces = detected_faces[i] if i < len(detected_faces) else []
            if not len(faces):
                resultados.append(None)
                continue
```

```
            preds = detector.get_landmarks_from_image(frame_rgb, detected_faces=faces)
        resultados.append(preds[0].astype(np.float32) if preds else None)
    return resultados
```

def normalize_lip_landmarks

server/main.py:157-176

```
def normalize_lip_landmarks(landmarks_px):
    right_corner = landmarks_px[MOUTH_RIGHT_CORNER]
    left_corner = landmarks_px[MOUTH_LEFT_CORNER]

    mouth_center = (right_corner + left_corner) / 2.0
    mouth_vector = left_corner - right_corner
    scale = np.linalg.norm(mouth_vector)
    if scale < 1e-3:
        return None
    angle = math.atan2(mouth_vector[1], mouth_vector[0])

    cos_a, sin_a = math.cos(-angle), math.sin(-angle)
    rotation = np.array([[cos_a, -sin_a], [sin_a, cos_a]], dtype=np.float64)

    lip_pts = landmarks_px[LIP_INDICES].astype(np.float64)
    translated = lip_pts - mouth_center
    rotated = translated @ rotation.T
    normalized = rotated / (scale + SCALE_EPSILON)

    return normalized.astype(np.float32).flatten()
```

def normalize_face_landmarks

server/main.py:179-192

```
def normalize_face_landmarks(landmarks_px):
    ojo_derecho = landmarks_px[OJO_DERECHO_EXTERNO].astype(np.float64)
    ojo_izquierdo = landmarks_px[OJO_IZQUIERDO_EXTERNO].astype(np.float64)
    centro = (ojo_derecho + ojo_izquierdo) / 2.0
    vector_ojos = ojo_izquierdo - ojo_derecho
    escala = np.linalg.norm(vector_ojos)
    if escala < 1e-3:
        return None
    angulo = math.atan2(vector_ojos[1], vector_ojos[0])
    cos_a, sin_a = math.cos(-angulo), math.sin(-angulo)
    rotacion = np.array([[cos_a, -sin_a], [sin_a, cos_a]], dtype=np.float64)
    puntos = landmarks_px[:68].astype(np.float64)
    normalizados = ((puntos - centro) @ rotacion.T) / (escala + SCALE_EPSILON)
    return normalizados.astype(np.float32).flatten()
```

def is_outlier

server/main.py:198-201

```
def is_outlier(vector, last_valid_vector):
    if last_valid_vector is None:
        return False
    return float(np.abs(vector - last_valid_vector).max()) > OUTLIER_MAX_JUMP
```

def fill_missing_frames

server/main.py:204-234

```
def fill_missing_frames(positions):
    primeros_validos = [i for i, p in enumerate(positions) if p is not None]
    if not primeros_validos:
        return None

    primer_valido = primeros_validos[0]
    ultimo_valido_idx = primeros_validos[-1]
    relleno = list(positions)

    for i in range(primer_valido):
        relleno[i] = relleno[primer_valido]

    i = primer_valido
    while i < ultimo_valido_idx:
        if relleno[i + 1] is None:
            j = i + 1
            while relleno[j] is None:
                j += 1
            inicio, fin = relleno[i], relleno[j]
            pasos = j - i
            for k in range(1, pasos):
                t = k / pasos
```

```
                relleno[i + k] = (1 - t) * inicio + t * fin
            i = j
        else:
            i += 1

    for i in range(ultimo_valido_idx + 1, len(relleno)):
        relleno[i] = relleno[ultimo_valido_idx]

    return np.stack(relleno, axis=0).astype(np.float32)
```

def smooth_positions

server/main.py:240-245

```
def smooth_positions(positions, alpha=SMOOTHING_ALPHA):
    smoothed = np.empty_like(positions)
    smoothed[0] = positions[0]
    for t in range(1, positions.shape[0]):
        smoothed[t] = alpha * positions[t] + (1 - alpha) * smoothed[t - 1]
    return smoothed.astype(np.float32)
```

def add_dynamics

server/main.py:248-255

```
def add_dynamics(positions):
    velocity = np.zeros_like(positions)
    velocity[1:] = positions[1:] - positions[:-1]

    acceleration = np.zeros_like(positions)
    acceleration[1:] = velocity[1:] - velocity[:-1]

    return np.concatenate([positions, velocity, acceleration], axis=1).astype(np.float32)
```

class PositionalEncoding

server/main.py:258-269

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=500):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, : x.size(1)]
```

class TCNBlock

server/main.py:272-291

```
class TCNBlock(nn.Module):
    def __init__(self, channels, kernel_size=3, dilation=1, dropout=0.1):
        super().__init__()
        padding = (kernel_size - 1) * dilation // 2
        self.conv1 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)
        self.bn1 = nn.BatchNorm1d(channels)
        self.conv2 = nn.Conv1d(channels, channels, kernel_size, padding=padding, dilation=dilation)
        self.bn2 = nn.BatchNorm1d(channels)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, key_padding_mask=None):
        residual = x
        x = x.transpose(1, 2)
        if key_padding_mask is not None:
            x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)
        x = self.dropout(self.relu(self.bn1(self.conv1(x))))
        x = self.bn2(self.conv2(x))
        x = x.transpose(1, 2)
        return self.relu(residual + self.dropout(x))
```

class TCNFeatureExtractor

server/main.py:294-305

```
class TCNFeatureExtractor(nn.Module):
    def __init__(self, channels, n_blocks=3, kernel_size=3, dropout=0.1):
        super().__init__()
        self.blocks = nn.ModuleList([
```

```
            TCNBlock(channels, kernel_size=kernel_size, dilation=2 ** i, dropout=dropout)
            for i in range(n_blocks)
        ])

    def forward(self, x, key_padding_mask=None):
        for block in self.blocks:
            x = block(x, key_padding_mask=key_padding_mask)
        return x
```

class ConformerFeedForward

server/main.py:308-321

```
class ConformerFeedForward(nn.Module):
    def __init__(self, d_model, dim_feedforward, dropout):
        super().__init__()
        self.net = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, dim_feedforward),
            nn.SiLU(),
            nn.Dropout(dropout),
            nn.Linear(dim_feedforward, d_model),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)
```

class ConformerConvModule

server/main.py:324-351

```
class ConformerConvModule(nn.Module):
    def __init__(self, d_model, kernel_size=15, dropout=0.1):
        super().__init__()
        self.layer_norm = nn.LayerNorm(d_model)
        self.pointwise_conv1 = nn.Conv1d(d_model, 2 * d_model, kernel_size=1)
        self.glu = nn.GLU(dim=1)
        padding = (kernel_size - 1) // 2
        self.depthwise_conv = nn.Conv1d(d_model, d_model, kernel_size=kernel_size, padding=padding, groups=d_model)
        self.batch_norm = nn.BatchNorm1d(d_model)
        self.swish = nn.SiLU()
        self.pointwise_conv2 = nn.Conv1d(d_model, d_model, kernel_size=1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, key_padding_mask=None):
        residual = x
        x = self.layer_norm(x)
        x = x.transpose(1, 2)
        if key_padding_mask is not None:
            x = x.masked_fill(key_padding_mask.unsqueeze(1), 0.0)
        x = self.pointwise_conv1(x)
        x = self.glu(x)
        x = self.depthwise_conv(x)
        x = self.batch_norm(x)
        x = self.swish(x)
        x = self.pointwise_conv2(x)
        x = self.dropout(x)
        x = x.transpose(1, 2)
        return residual + x
```

class ConformerBlock

server/main.py:354-373

```
class ConformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, dim_feedforward, conv_kernel_size=15, dropout=0.1):
        super().__init__()
        self.ff1 = ConformerFeedForward(d_model, dim_feedforward, dropout)
        self.self_attn_norm = nn.LayerNorm(d_model)
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
        self.attn_dropout = nn.Dropout(dropout)
        self.conv_module = ConformerConvModule(d_model, conv_kernel_size, dropout)
        self.ff2 = ConformerFeedForward(d_model, dim_feedforward, dropout)
        self.final_norm = nn.LayerNorm(d_model)

    def forward(self, x, src_key_padding_mask=None):
        x = x + 0.5 * self.ff1(x)
        residual = x
        x_norm = self.self_attn_norm(x)
        attn_out, _ = self.self_attn(x_norm, x_norm, x_norm, key_padding_mask=src_key_padding_mask, need_weights=False)
        x = residual + self.attn_dropout(attn_out)
        x = self.conv_module(x, key_padding_mask=src_key_padding_mask)
```

```
        x = x + 0.5 * self.ff2(x)
        return self.final_norm(x)
```

class ConformerEncoder

server/main.py:376-387

```
class ConformerEncoder(nn.Module):
    def __init__(self, d_model, n_layers, n_heads, dim_feedforward, conv_kernel_size=15, dropout=0.1):
        super().__init__()
        self.layers = nn.ModuleList([
            ConformerBlock(d_model, n_heads, dim_feedforward, conv_kernel_size, dropout)
            for _ in range(n_layers)
        ])

    def forward(self, x, src_key_padding_mask=None):
        for layer in self.layers:
            x = layer(x, src_key_padding_mask=src_key_padding_mask)
        return x
```

def masked_mean_pool

server/main.py:390-396

```
def masked_mean_pool(x, src_key_padding_mask):
    if src_key_padding_mask is None:
        return x.mean(dim=1)
    real_mask = (~src_key_padding_mask).unsqueeze(-1).float()
    summed = (x * real_mask).sum(dim=1)
    counts = real_mask.sum(dim=1).clamp(min=1.0)
    return summed / counts
```

class LipReadingConformer

server/main.py:399-421

```
class LipReadingConformer(nn.Module):
    def __init__(self, num_classes, input_dim, hidden_dim=128,
                 n_layers=2, n_heads=8, dim_feedforward=256,
                 conv_kernel_size=15, tcn_blocks=3, tcn_kernel_size=3, dropout=0.3):
        super().__init__()
        self.input_proj = nn.Linear(input_dim, hidden_dim)
        self.tcn = TCNFeatureExtractor(hidden_dim, n_blocks=tcn_blocks, kernel_size=tcn_kernel_size, dropout=dropout)
        self.pos_encoder = PositionalEncoding(hidden_dim)
        self.conformer_encoder = ConformerEncoder(
            d_model=hidden_dim, n_layers=n_layers, n_heads=n_heads,
            dim_feedforward=dim_feedforward, conv_kernel_size=conv_kernel_size, dropout=dropout,
        )
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, 64), nn.ReLU(), nn.Dropout(dropout), nn.Linear(64, num_classes),
        )

    def forward(self, x, src_key_padding_mask=None):
        x = self.input_proj(x)
        x = self.tcn(x, key_padding_mask=src_key_padding_mask)
        x = self.pos_encoder(x)
        x = self.conformer_encoder(x, src_key_padding_mask=src_key_padding_mask)
        pooled = masked_mean_pool(x, src_key_padding_mask)
        return self.classifier(pooled)
```

def make_padding_mask

server/main.py:424-426

```
def make_padding_mask(lengths, max_len, device):
    idx = torch.arange(max_len, device=device).unsqueeze(0)
    return idx >= lengths.unsqueeze(1)
```

def safe_torch_load

server/main.py:429-433

```
def safe_torch_load(path):
    try:
        return torch.load(path, weights_only=True)
    except Exception:
        return torch.load(path, weights_only=False)
```

def load_everything

server/main.py:452-551

```
@app.on_event("startup")
async def load_everything():
```

```
db.init_db()

if not os.path.exists(MODELO_CARA_PATH):
    print(f"[AVISO] No encontré {MODELO_CARA_PATH} -- /predict queda deshabilitado. "
          "Entrenalo con Colab/Entrenar_Cara_Completa_FonoKids.ipynb. "
          "El resto del server (dashboard, stats) funciona igual.")
    return

ckpt_cara = safe_torch_load(MODELO_CARA_PATH)
class_names = ckpt_cara["class_names"]
max_frames = ckpt_cara["max_frames"]
modelo_cara = LipReadingConformer(num_classes=len(class_names), input_dim=ckpt_cara["input_dim"]).to(DEVICE)
modelo_cara.load_state_dict(ckpt_cara["model_state_dict"])
modelo_cara.eval()
print(f"Modelo de cara completa cargado ({len(class_names)} clases, entrada {ckpt_cara['input_dim']}, "
      f"val_acc guardado: {ckpt_cara.get('val_acc', '?')}).")

# [LABIOS DESACTIVADO] if not os.path.exists(MODEL_PATH):
    # [LABIOS DESACTIVADO] print(f"[AVISO] No encontré {MODEL_PATH} -- /predict queda deshabilitado. "
          # [LABIOS DESACTIVADO] "Entrenalo con train_landmarks_transformer.py o bajalo de Colab. "
          # [LABIOS DESACTIVADO] "El resto del server (dashboard, stats) funciona igual.")
    # [LABIOS DESACTIVADO] return

# [LABIOS DESACTIVADO] checkpoint = safe_torch_load(MODEL_PATH)
# [LABIOS DESACTIVADO] class_names = checkpoint["class_names"]
# [LABIOS DESACTIVADO] max_frames = checkpoint["max_frames"]
# [LABIOS DESACTIVADO] input_dim = checkpoint.get("input_dim", 120)

# [LABIOS DESACTIVADO] model = LipReadingConformer(num_classes=len(class_names), input_dim=input_dim).to(DEVICE)
# [LABIOS DESACTIVADO] model.load_state_dict(checkpoint["model_state_dict"])
# [LABIOS DESACTIVADO] model.eval()

print("Cargando detector de landmarks (HRNet/FAN)...")
landmarker = build_hrnet_detector()

if DEVICE == "cuda":
    try:
        print("Precalentando HRNet (primera compilación, puede tardar)...")
        dummy_frames = [np.zeros((240, 320, 3), dtype=np.uint8) for _ in range(3)]
        detect_landmarks_batch(landmarker, dummy_frames)
        print("Precalentamiento listo.")
    except Exception as e:
        print(f"[AVISO] Precalentamiento falló ({e}), no es grave -- "
              "la primera predicción real puede tardar más de lo normal.")

# [LABIOS DESACTIVADO] _state.update(model=model, landmarker=landmarker, class_names=class_names, max_frames=max_frames)
_state.update(landmarker=landmarker, model_cara=modelo_cara, class_names_cara=class_names, max_frames_cara=max_frames)

# [LABIOS DESACTIVADO] if USAR_MODELO_CARA and os.path.exists(MODELO_CARA_PATH):
    # [LABIOS DESACTIVADO] try:
        # [LABIOS DESACTIVADO] ckpt_cara = safe_torch_load(MODELO_CARA_PATH)
        # [LABIOS DESACTIVADO] modelo_cara = LipReadingConformer(
            # [LABIOS DESACTIVADO] num_classes=len(ckpt_cara["class_names"]), input_dim=ckpt_cara["input_dim"],
        # [LABIOS DESACTIVADO] ).to(DEVICE)
        # [LABIOS DESACTIVADO] modelo_cara.load_state_dict(ckpt_cara["model_state_dict"])
        # [LABIOS DESACTIVADO] modelo_cara.eval()
        # [LABIOS DESACTIVADO] _state.update(model_cara=modelo_cara, class_names_cara=ckpt_cara["class_names"],
                    # [LABIOS DESACTIVADO] max_frames_cara=ckpt_cara["max_frames"])
        # [LABIOS DESACTIVADO] print(f"Modelo de cara completa cargado ({len(ckpt_cara['class_names'])} clases, "
              # [LABIOS DESACTIVADO] f"entrada {ckpt_cara['input_dim']}, val_acc guardado: {ckpt_cara.get('val_acc', '?')}).")
    # [LABIOS DESACTIVADO] except Exception as e:
        # [LABIOS DESACTIVADO] print(f"[AVISO] No se pudo cargar el modelo de cara completa ({e}) -- sigo solo con el de labios.")
# [LABIOS DESACTIVADO] else:
    # [LABIOS DESACTIVADO] print("Modelo de cara completa desactivado o no encontrado -- se usa solo el de labios.")
# [LABIOS DESACTIVADO] print(f"Listo. Device: {DEVICE}. Clases ({len(class_names)}): {class_names}")
print(f"Listo. Device: {DEVICE}. Solo cara completa. Clases ({len(class_names)}): {class_names}")

try:
    print("Cargando modelo de audio (wav2vec2, puede tardar la primera vez)...")
    audio_model, audio_fe, audio_vocab, _ = load_audio_model(device=DEVICE)
    _state.update(audio_model=audio_model, audio_feature_extractor=audio_fe, audio_vocab=audio_vocab)
    print("Modelo de audio listo.")
except Exception as e:
    print(f"[AVISO] No se pudo cargar el modelo de audio ({e}) -- "
          "el veredicto va a usar solo el video, sin análisis de sonido.")

if os.path.exists(AUDIO_CLASSIFIER_PATH):
    try:
        ckpt = safe_torch_load(AUDIO_CLASSIFIER_PATH)
        clf = AudioClassifierHead(
            ckpt["embedding_dim"], ckpt["hidden_dim"], len(ckpt["class_names"]), dropout=0.0,
        ).to(DEVICE)
        clf.load_state_dict(ckpt["state_dict"])
```

```
            clf.eval()
            _state.update(
                audio_clf=clf,
                audio_clf_classes=ckpt["class_names"],
                audio_clf_mean=ckpt["mean"],
                audio_clf_std=ckpt["std"],
            )
            print(f"Clasificador de audio entrenado cargado ({len(ckpt['class_names'])} clases, "
                  f"val_acc guardado: {ckpt.get('val_acc', '?')}).")
        except Exception as e:
            print(f"[AVISO] No se pudo cargar el clasificador de audio entrenado ({e}) -- "
                  "sigo solo con el GOP genérico.")
    else:
        print(f"[AVISO] No hay clasificador de audio entrenado todavía ({AUDIO_CLASSIFIER_PATH} "
              "no existe) -- corré train_audio_classifier.py cuando tengas datos suficientes.")
```

def cleanup

server/main.py:554-558

```
@app.on_event("shutdown")
def cleanup():
    landmarker = _state["landmarker"]
    if landmarker is not None and hasattr(landmarker, "close"):
        landmarker.close()
```

def preprocess_sequence

server/main.py:561-579

```
def preprocess_sequence(positions, max_frames):
    smoothed = smooth_positions(positions)
    sequence_np = add_dynamics(smoothed)
    sequence = torch.from_numpy(sequence_np).float()

    T = sequence.shape[0]
    if T < max_frames:
        real_length = T
        padding = torch.zeros((max_frames - T,) + sequence.shape[1:], dtype=sequence.dtype)
        sequence = torch.cat([sequence, padding], dim=0)
    elif T > max_frames:
        start = (T - max_frames) // 2
        sequence = sequence[start:start + max_frames]
        real_length = max_frames
    else:
        real_length = T

    mask = make_padding_mask(torch.tensor([real_length]), max_frames, sequence.device)
    return sequence.unsqueeze(0), mask
```

def health

server/main.py:582-585

```
@app.get("/health")
def health():
    return {"status": "ok", "device": DEVICE, "classes": _state["class_names_cara"],
            "cara_completa": _state["model_cara"] is not None}
```

def _count_files

server/main.py:591-594

```
def _count_files(folder, extensions=VIDEO_EXTENSIONS):
    if not os.path.isdir(folder):
        return 0
    return len([f for f in os.listdir(folder) if f.lower().endswith(extensions)])
```

def dataset_stats

server/main.py:597-620

```
@app.get("/dataset/stats")
def dataset_stats():
    class_names_dirs = set()
    if os.path.isdir(SESSIONS_DIR):
        class_names_dirs.update(d for d in os.listdir(SESSIONS_DIR)
                                if os.path.isdir(os.path.join(SESSIONS_DIR, d)))

    rows = []
    total_clips = 0
    for class_name in sorted(class_names_dirs):
        n_clips = _count_files(os.path.join(SESSIONS_DIR, class_name, "clips"))
        palabra, correcta, tipo_error = _parse_clase_predicha(class_name)
```

```
        rows.append({
            "class_name": class_name, "palabra": palabra, "correcta": correcta,
            "tipo_error": tipo_error, "clips": n_clips,
        })
        total_clips += n_clips

    return {
        "goal_per_word": GOAL_PER_WORD,
        "total_words": len(rows),
        "total_clips": total_clips,
        "words": rows,
    }
```

def _parse_clip_filename

server/main.py:623-630

```
def _parse_clip_filename(class_name, filename):
    stem = os.path.splitext(filename)[0]
    prefix = class_name + "_"
    rest = stem[len(prefix):] if stem.lower().startswith(prefix.lower()) else stem
    if "_" in rest:
        person, _seq = rest.rsplit("_", 1)
        return person
    return None
```

def dataset_recordings

server/main.py:633-673

```
@app.get("/dataset/recordings")
def dataset_recordings():
    from_db = db.get_recordings_grouped()
    if from_db is not None:
        return {"recordings": from_db}

    if not os.path.isdir(SESSIONS_DIR):
        return {"recordings": []}

    grouped = defaultdict(list)
    for class_name in os.listdir(SESSIONS_DIR):
        clips_dir = os.path.join(SESSIONS_DIR, class_name, "clips")
        if not os.path.isdir(clips_dir):
            continue
        for filename in os.listdir(clips_dir):
            if not filename.lower().endswith(VIDEO_EXTENSIONS):
                continue
            path = os.path.join(clips_dir, filename)
            person = _parse_clip_filename(class_name, filename) or "desconocido"
            mtime = os.path.getmtime(path)
            date = datetime.fromtimestamp(mtime).strftime("%Y-%m-%d")
            grouped[(date, class_name, person)].append((mtime, filename))

    recordings = []
    for (date, class_name, person), files in grouped.items():
        files.sort(key=lambda f: f[0], reverse=True)
        palabra, correcta, tipo_error = _parse_clase_predicha(class_name)
        recordings.append({
            "date": date,
            "time": datetime.fromtimestamp(files[0][0]).strftime("%H:%M"),
            "class_name": class_name,
            "word": palabra,
            "correcta": correcta,
            "tipo_error": tipo_error,
            "person": person,
            "count": len(files),
            "thumbnail_file": files[0][1],
        })
    recordings.sort(key=lambda r: r["date"], reverse=True)

    return {"recordings": recordings}
```

def dataset_thumbnail

server/main.py:676-704

```
@app.get("/dataset/thumbnail")
def dataset_thumbnail(word: str, filename: str):
    safe_filename = os.path.basename(filename)
    safe_word = os.path.basename(word)
    path = os.path.join(SESSIONS_DIR, safe_word, "clips", safe_filename)
    if not os.path.isfile(path):
        raise HTTPException(status_code=404, detail="Clip no encontrado.")
```

```
    cap = cv2.VideoCapture(path)
    if not cap.isOpened():
        cap.release()
        raise HTTPException(status_code=422, detail="No se pudo leer el clip.")

    frames = []
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        frames.append(frame)
    cap.release()

    if not frames:
        raise HTTPException(status_code=422, detail="No se pudo leer ningún frame del clip.")
    frame = frames[len(frames) // 2]

    ok, buffer = cv2.imencode(".jpg", frame)
    if not ok:
        raise HTTPException(status_code=500, detail="No se pudo codificar la imagen.")
    return Response(content=buffer.tobytes(), media_type="image/jpeg")
```

def _pick_reference_clip

server/main.py:710-725

```
def _pick_reference_clip(palabra):
    clips_dir = os.path.join(SESSIONS_DIR, f"{palabra}_correcto", "clips")
    if not os.path.isdir(clips_dir):
        return None
    candidatos = [f for f in os.listdir(clips_dir) if f.lower().endswith(VIDEO_EXTENSIONS)]
    if not candidatos:
        return None

    def _n_frames(filename):
        cap = cv2.VideoCapture(os.path.join(clips_dir, filename))
        n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        cap.release()
        return n

    mejor = max(candidatos, key=_n_frames)
    return os.path.join(clips_dir, mejor), mejor
```

def reference_landmarks

server/main.py:728-781

```
@app.get("/reference/{palabra}")
def reference_landmarks(palabra: str):
    if _state["landmarker"] is None:
        raise HTTPException(status_code=503, detail="El servidor todavía está cargando el modelo.")

    safe_palabra = os.path.basename(palabra)
    elegido = _pick_reference_clip(safe_palabra)
    if elegido is None:
        raise HTTPException(
            status_code=404,
            detail=f"No hay clips grabados en {safe_palabra}_correcto/clips/ todavía.",
        )
    clip_path, filename = elegido

    cached = _reference_cache.get(safe_palabra)
    if cached is not None and cached["filename"] == filename:
        return cached["response"]

    cap = cv2.VideoCapture(clip_path)
    if not cap.isOpened():
        cap.release()
        raise HTTPException(status_code=422, detail="No se pudo leer el clip de referencia.")
    frames = []
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        h, w = frame.shape[:2]
        if w > MAX_FRAME_WIDTH:
            scale = MAX_FRAME_WIDTH / w
            frame = cv2.resize(frame, (MAX_FRAME_WIDTH, round(h * scale)), interpolation=cv2.INTER_AREA)
        frames.append(frame)
    cap.release()

    landmarks_por_frame = detect_landmarks_batch(_state["landmarker"], frames)
```

```
    positions = []
    last_valid_vector = None
    for landmarks_px in landmarks_por_frame:
        vector = normalize_lip_landmarks(landmarks_px) if landmarks_px is not None else None
        if vector is not None and is_outlier(vector, last_valid_vector):
            vector = None
        if vector is not None:
            last_valid_vector = vector
        positions.append(vector)

    filled = fill_missing_frames(positions)
    if filled is None:
        raise HTTPException(status_code=422, detail="No se detectó ninguna cara en el clip de referencia.")
    smoothed = smooth_positions(filled)

    frames_out = [smoothed[t].reshape(20, 2).tolist() for t in range(smoothed.shape[0])]
    response = {"palabra": safe_palabra, "frames": frames_out}
    _reference_cache[safe_palabra] = {"filename": filename, "response": response}
    return response
```

def _guardar_debug_frase

server/main.py:788-804

```
def _guardar_debug_frase(audio_path, recortado_path, palabra, motivo):
    try:
        os.makedirs(DEBUG_FRASES_DIR, exist_ok=True)
        marca = time.strftime("%Y%m%d_%H%M%S")
        base = os.path.join(DEBUG_FRASES_DIR, f"{marca}_{palabra}_{motivo}")
        if audio_path and os.path.exists(audio_path):
            shutil.copyfile(audio_path, base + "_completo.wav")
        if recortado_path and os.path.exists(recortado_path):
            shutil.copyfile(recortado_path, base + "_recorte.wav")
        archivos = sorted(
            (os.path.join(DEBUG_FRASES_DIR, f) for f in os.listdir(DEBUG_FRASES_DIR)),
            key=os.path.getmtime,
        )
        for viejo in archivos[:-DEBUG_FRASES_MAX_ARCHIVOS]:
            os.remove(viejo)
    except Exception as e:
        print(f"[AVISO] No se pudo guardar el audio de depuracion ({e}).")
```

def _save_practice_clip

server/main.py:807-816

```
def _save_practice_clip(class_name, tmp_path, suffix):
    try:
        clips_dir = os.path.join(SESSIONS_DIR, class_name, "clips")
        os.makedirs(clips_dir, exist_ok=True)
        dest_name = f"{class_name}_web_{int(time.time() * 1000)}{suffix}"
        shutil.copy(tmp_path, os.path.join(clips_dir, dest_name))
        return dest_name
    except OSError as e:
        print(f"[AVISO] No se pudo guardar el clip de práctica ({e}).")
        return None
```

def _score_audio_pronunciation

server/main.py:822-834

```
def _score_audio_pronunciation(audio_path, palabra):
    if audio_path is None or not palabra:
        return None
    if _state["audio_model"] is None:
        return None
    try:
        return score_pronunciation(
            audio_path, palabra,
            _state["audio_model"], _state["audio_feature_extractor"], _state["audio_vocab"], DEVICE,
        )
    except Exception as e:
        print(f"[AVISO] Falló el análisis de audio ({e}) -- sigo solo con video.")
        return None
```

def _score_word_in_sentence

server/main.py:837-849

```
def _score_word_in_sentence(audio_path, frase, palabra):
    if audio_path is None or not frase or not palabra:
        return None
    if _state["audio_model"] is None:
```

```
        return None
    try:
        return score_word_in_sentence(
            audio_path, frase, palabra,
            _state["audio_model"], _state["audio_feature_extractor"], _state["audio_vocab"], DEVICE,
        )
    except Exception as e:
        print(f"[AVISO] Falló el análisis de frase ({e}).")
        return None
```

def _classify_audio

server/main.py:852-880

```
def _classify_audio(audio_path, palabra):
    clf = _state["audio_clf"]
    if clf is None or audio_path is None or not palabra:
        return None
    class_names = _state["audio_clf_classes"]
    indices_de_la_palabra = [
        i for i, c in enumerate(class_names)
        if c == f"{palabra}_correcto" or c.startswith(f"{palabra}_incorrecto")
    ]
    if not indices_de_la_palabra:
        return None
    try:
        emb = extract_embedding(audio_path, _state["audio_model"], _state["audio_feature_extractor"], DEVICE)
        emb_norm = (emb - _state["audio_clf_mean"]) / _state["audio_clf_std"]
        x = torch.from_numpy(emb_norm.reshape(1, -1)).float().to(DEVICE)
        with torch.no_grad():
            probs = torch.softmax(clf(x), dim=1)[0]
        probs_familia = probs[indices_de_la_palabra]
        idx_local = int(torch.argmax(probs_familia).item())
        idx = indices_de_la_palabra[idx_local]
        clase_predicha = class_names[idx]
        return {
            "clase_predicha": clase_predicha,
            "prob": float(probs[idx].item()),
            "es_correcto": clase_predicha == f"{palabra}_correcto",
        }
    except Exception as e:
        print(f"[AVISO] Falló el clasificador de audio entrenado ({e}) -- sigo solo con el GOP.")
        return None
```

def _run_audio_only_pipeline

server/main.py:883-954

```
def _run_audio_only_pipeline(audio_path, palabra_esperada):
    t_start = time.time()

    es_palabra_completa = palabra_esperada and "_" not in palabra_esperada
    audio_resultado = (
        _score_audio_pronunciation(audio_path, palabra_esperada) if es_palabra_completa else None
    )
    audio_ok = (
        audio_resultado is not None
        and len(audio_resultado["sospechosos"]) == 0
        and audio_resultado["palabra_reconocida"]
    )
    audio_clf_resultado = _classify_audio(audio_path, palabra_esperada)
    prob_str = f"{audio_clf_resultado['prob']:.2f}" if audio_clf_resultado else "?"
    print(f"[predict-audio] audio GOP {'OK' if audio_resultado is None else ('bien' if audio_ok else 'sospechoso')} | "
          f"clasificador {'sin entrenar' if audio_clf_resultado is None else audio_clf_resultado['clase_predicha']} "
          f"prob={prob_str} (umbral {AUDIO_MIN_CONFIDENCE}) "
          f"({time.time() - t_start:.1f}s total)")

    if audio_clf_resultado is None:
        return {
            "valid": False,
            "reason": "no se pudo analizar el audio -- probá de nuevo",
        }

    audio_confiable = audio_clf_resultado["prob"] >= AUDIO_MIN_CONFIDENCE
    audio_dice_bien = audio_clf_resultado["es_correcto"]

    clase_predicha = audio_clf_resultado["clase_predicha"]
    palabra, _pronunciacion_correcta, tipo_error = _parse_clase_predicha(clase_predicha)

    if not audio_confiable:
        veredicto_final = "no_concluyente"
    else:
        veredicto_final = "bien" if audio_dice_bien else "a_practicar"
```

```
    if tipo_error is None and veredicto_final == "a_practicar" and audio_resultado is not None:
        tipo_error = audio_resultado.get("tipo_error_audio")

    confianza_suficiente = audio_confiable

    if confianza_suficiente:
        db.save_attempt(
            class_name=clase_predicha, palabra=palabra, correcta=(veredicto_final == "bien"),
            tipo_error=tipo_error if veredicto_final == "a_practicar" else None,
            prob=audio_clf_resultado["prob"],
        )

    return {
        "valid": True,
        "palabra": palabra,
        "correcta": (veredicto_final == "bien") if confianza_suficiente else None,
        "veredicto_final": veredicto_final,
        "audio": {
            "fonemas": audio_resultado["fonemas"],
            "scores": audio_resultado["scores"],
            "sospechosos": audio_resultado["sospechosos"],
            "reconocido_libre": audio_resultado["reconocido_libre"],
            "sonido_reconocido": audio_resultado["sonido_reconocido"],
            "palabra_reconocida": audio_resultado["palabra_reconocida"],
        } if audio_resultado is not None else None,
        "audio_clasificador": audio_clf_resultado,
        "tipo_error": tipo_error if (confianza_suficiente and veredicto_final == "a_practicar") else None,
        "palabra_distinta": False,
        "class_name": clase_predicha,
        "prob": audio_clf_resultado["prob"],
        "detected": None,
        "total": None,
        "confianza_suficiente": confianza_suficiente,
        "capture_file": None,
        "all_probs": None,
    }
```

def _run_prediction_pipeline

server/main.py:957-1200

```
def _run_prediction_pipeline(tmp_path, suffix, landmarker, model, class_names, max_frames,
                             audio_path=None, palabra_esperada=None):
    t_start = time.time()
    if DEVICE == "cuda":
        torch.cuda.empty_cache()
    cap = cv2.VideoCapture(tmp_path)
    if not cap.isOpened():
        raise HTTPException(status_code=400, detail="No se pudo leer el video enviado.")

    frames = []
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        h, w = frame.shape[:2]
        if w > MAX_FRAME_WIDTH:
            scale = MAX_FRAME_WIDTH / w
            frame = cv2.resize(frame, (MAX_FRAME_WIDTH, round(h * scale)), interpolation=cv2.INTER_AREA)
        frames.append(frame)
    cap.release()
    print(f"[predict] frames leídos: {len(frames)} ({time.time() - t_start:.1f}s)")

    total = len(frames)
    if total == 0:
        return {
            "valid": False,
            "reason": "no se pudo leer ningún frame del video -- el formato/códec "
                      "puede no ser compatible con el servidor (probá grabar en H.264/mp4)",
        }
    if total < MIN_FRAMES_VALID:
        return {
            "valid": False,
            "reason": f"toma muy corta ({total} frames, minimo {MIN_FRAMES_VALID}) "
                      "-- probablemente no se dijo la frase completa",
        }

    modelo_video = "cara_completa"
    if palabra_esperada is not None and not (
            f"{palabra_esperada}_correcto" in class_names and f"{palabra_esperada}_incorrecto" in class_names):
        # La palabra no está en el modelo de cara completa (hoy: perro): se evalúa solo con el audio.
        print(f"[predict] '{palabra_esperada}' no está en el modelo de cara completa -> solo audio")
```

```
        if audio_path is not None:
            return _run_audio_only_pipeline(audio_path, palabra_esperada)
        return {"valid": False, "reason": f"no hay modelo de video para '{palabra_esperada}' -- mandá también el audio"}

    landmarks_por_frame = detect_landmarks_batch(landmarker, frames)
    print(f"[predict] landmarks detectados ({time.time() - t_start:.1f}s)")

    positions = []
    detected = 0
    last_valid_vector = None
    for landmarks_px in landmarks_por_frame:
        vector = normalize_face_landmarks(landmarks_px) if landmarks_px is not None else None
        if vector is not None and is_outlier(vector, last_valid_vector):
            vector = None
        if vector is not None:
            detected += 1
            last_valid_vector = vector
        positions.append(vector)

    detection_rate = detected / total if total else 0
    filled = fill_missing_frames(positions)
    if filled is None or detection_rate < MIN_DETECCION_CARA:
        return {
            "valid": False,
            "reason": f"cara entera detectada solo en {detected}/{total} frames "
                      f"({detection_rate*100:.0f}%) -- encuadre malo, repetí la toma",
        }

    # [LABIOS DESACTIVADO] positions = []
    # [LABIOS DESACTIVADO] detected = 0
    # [LABIOS DESACTIVADO] last_valid_vector = None
    # [LABIOS DESACTIVADO] for landmarks_px in landmarks_por_frame:
        # [LABIOS DESACTIVADO] vector = normalize_lip_landmarks(landmarks_px) if landmarks_px is not None else None
        # [LABIOS DESACTIVADO] if vector is not None and is_outlier(vector, last_valid_vector):
            # [LABIOS DESACTIVADO] vector = None
        # [LABIOS DESACTIVADO] if vector is not None:
            # [LABIOS DESACTIVADO] detected += 1
            # [LABIOS DESACTIVADO] last_valid_vector = vector
        # [LABIOS DESACTIVADO] positions.append(vector)

    # [LABIOS DESACTIVADO] detection_rate = detected / total if total else 0
    # [LABIOS DESACTIVADO] filled = fill_missing_frames(positions)
    # [LABIOS DESACTIVADO] if filled is None or detection_rate < MIN_DETECTION_RATE_VALID:
        # [LABIOS DESACTIVADO] return {
            # [LABIOS DESACTIVADO] "valid": False,
            # [LABIOS DESACTIVADO] "reason": f"cara detectada solo en {detected}/{total} frames "
                      # [LABIOS DESACTIVADO] f"({detection_rate*100:.0f}%) -- encuadre malo, repetí la toma",
        # [LABIOS DESACTIVADO] }

    # [LABIOS DESACTIVADO] modelo_video = "labios"
    # [LABIOS DESACTIVADO] clases_cara = _state["class_names_cara"]
    # [LABIOS DESACTIVADO] if (_state["model_cara"] is not None and palabra_esperada is not None
            # [LABIOS DESACTIVADO] and f"{palabra_esperada}_correcto" in clases_cara and f"{palabra_esperada}_incorrecto" in
clases_cara):
        # [LABIOS DESACTIVADO] posiciones_cara = []
        # [LABIOS DESACTIVADO] detectados_cara = 0
        # [LABIOS DESACTIVADO] ultimo_cara = None
        # [LABIOS DESACTIVADO] for landmarks_px in landmarks_por_frame:
            # [LABIOS DESACTIVADO] vector_cara = normalize_face_landmarks(landmarks_px) if landmarks_px is not None else None
            # [LABIOS DESACTIVADO] if vector_cara is not None and is_outlier(vector_cara, ultimo_cara):
                # [LABIOS DESACTIVADO] vector_cara = None
            # [LABIOS DESACTIVADO] if vector_cara is not None:
                # [LABIOS DESACTIVADO] detectados_cara += 1
                # [LABIOS DESACTIVADO] ultimo_cara = vector_cara
            # [LABIOS DESACTIVADO] posiciones_cara.append(vector_cara)
        # [LABIOS DESACTIVADO] tasa_cara = detectados_cara / total
        # [LABIOS DESACTIVADO] relleno_cara = fill_missing_frames(posiciones_cara)
        # [LABIOS DESACTIVADO] if relleno_cara is not None and tasa_cara >= MIN_DETECCION_CARA:
            # [LABIOS DESACTIVADO] filled = relleno_cara
            # [LABIOS DESACTIVADO] model = _state["model_cara"]
            # [LABIOS DESACTIVADO] class_names = clases_cara
            # [LABIOS DESACTIVADO] max_frames = _state["max_frames_cara"]
            # [LABIOS DESACTIVADO] modelo_video = "cara_completa"
        # [LABIOS DESACTIVADO] else:
            # [LABIOS DESACTIVADO] print(f"[predict] cara completa no usable ({detectados_cara}/{total} frames con la cara entera): uso
el modelo de labios")
    sequence, mask = preprocess_sequence(filled, max_frames)
    sequence, mask = sequence.to(DEVICE), mask.to(DEVICE)
    with torch.inference_mode():
        logits = model(sequence, src_key_padding_mask=mask)
        probs = torch.softmax(logits, dim=1)[0]
    print(f"[predict] modelo Conformer ({modelo_video}) listo ({time.time() - t_start:.1f}s total)")
```

```
    sorted_probs, sorted_idx = torch.sort(probs, descending=True)
    top_idx = int(sorted_idx[0].item())
    top_prob = float(sorted_probs[0].item())
    second_prob = float(sorted_probs[1].item()) if len(sorted_probs) > 1 else 0.0
    margin = top_prob - second_prob
    clase_predicha = class_names[top_idx]
    print(f"[predict] video[{modelo_video}]: clase={clase_predicha} prob={top_prob:.2f} margin={margin:.2f} "
          f"(2da opcion: {class_names[int(sorted_idx[1].item())] if len(sorted_idx) > 1 else '?'} "
          f"{second_prob:.2f}) | esperado={palabra_esperada}")
    palabra, pronunciacion_correcta, tipo_error = _parse_clase_predicha(clase_predicha)

    def _palabra_base(nombre):
        return nombre.split("_")[0]

    palabra_distinta = False
    if palabra_esperada is not None and palabra != palabra_esperada:
        pidio_palabra_completa = palabra_esperada == _palabra_base(palabra_esperada)
        misma_palabra_base = _palabra_base(palabra) == _palabra_base(palabra_esperada)
        if not (pidio_palabra_completa and misma_palabra_base):
            pronunciacion_correcta = False
            tipo_error = None
            palabra_distinta = True

    all_probs = {class_names[i]: float(probs[i].item()) for i in range(len(class_names))}

    es_palabra_completa = (palabra_esperada or palabra) and "_" not in (palabra_esperada or palabra)
    audio_resultado = (
        _score_audio_pronunciation(audio_path, palabra_esperada or palabra)
        if es_palabra_completa else None
    )
    audio_ok = (
        audio_resultado is not None
        and len(audio_resultado["sospechosos"]) == 0
        and audio_resultado["palabra_reconocida"]
    )

    audio_clf_resultado = _classify_audio(audio_path, palabra_esperada or palabra)
    prob_str = f"{audio_clf_resultado['prob']:.2f}" if audio_clf_resultado else "?"
    print(f"[predict] audio GOP {'OK' if audio_resultado is None else ('bien' if audio_ok else 'sospechoso')} | "
          f"clasificador {'sin entrenar' if audio_clf_resultado is None else audio_clf_resultado['clase_predicha']} "
          f"prob={prob_str} (umbral {AUDIO_MIN_CONFIDENCE}) video_prob={top_prob:.2f} "
          f"({time.time() - t_start:.1f}s total)")

    acoustic_match = (
        audio_clf_resultado is not None
        and audio_clf_resultado["prob"] >= ACOUSTIC_MATCH_THRESHOLD
        and audio_clf_resultado["es_correcto"]
    )
    visual_match = top_prob >= VISUAL_MATCH_THRESHOLD and pronunciacion_correcta

    confianza_suficiente = audio_clf_resultado is not None

    audio_gop_ok = (
        audio_resultado is not None
        and not audio_resultado["sospechosos"]
        and audio_resultado["palabra_reconocida"]
    )
    audio_confirma = (
        audio_clf_resultado is not None
        and audio_clf_resultado["es_correcto"]
        and (audio_clf_resultado["prob"] >= AUDIO_MIN_CONFIDENCE
             or (audio_gop_ok and audio_clf_resultado["prob"] >= AUDIO_CONFIRMA_CON_GOP))
    )
    video_dice_mal_misma_palabra = (
        (not palabra_distinta) and top_prob >= VISUAL_MATCH_THRESHOLD and not pronunciacion_correcta
    )

    if not confianza_suficiente:
        veredicto_final = "no_concluyente"
        tipo_error = None
    elif acoustic_match and visual_match:
        veredicto_final = "bien"
        tipo_error = None
    elif audio_confirma and not video_dice_mal_misma_palabra:
        veredicto_final = "bien"
        tipo_error = None
        print(f"[predict] audio confirma '{palabra_esperada}' (clasificador {audio_clf_resultado['prob']:.2f}, "
              f"GOP {'bien' if audio_gop_ok else 'dudoso'}); el video ({palabra}, {top_prob:.2f}) no lo contradice "
              f"con seguridad -> bien")
    elif (not acoustic_match) and visual_match:
        veredicto_final = "a_practicar"
        tipo_error = "sustitucion_acustica"
    elif (not acoustic_match) and (not visual_match):
```

```
        veredicto_final = "a_practicar"
        tipo_error = "motor_severo"
    else:
        veredicto_final = "a_practicar"
        tipo_error = "compensacion_visual"

    capture_file = None
    if confianza_suficiente:
        capture_file = _save_practice_clip(clase_predicha, tmp_path, suffix)
        db.save_attempt(
            class_name=clase_predicha, palabra=palabra, correcta=(veredicto_final == "bien"),
            tipo_error=tipo_error if veredicto_final == "a_practicar" else None, prob=top_prob,
        )

    return {
        "valid": True,
        "palabra": palabra,
        "correcta": (veredicto_final == "bien") if confianza_suficiente else None,
        "veredicto_final": veredicto_final,
        "audio": {
            "fonemas": audio_resultado["fonemas"],
            "scores": audio_resultado["scores"],
            "sospechosos": audio_resultado["sospechosos"],
            "reconocido_libre": audio_resultado["reconocido_libre"],
            "sonido_reconocido": audio_resultado["sonido_reconocido"],
            "palabra_reconocida": audio_resultado["palabra_reconocida"],
        } if audio_resultado is not None else None,
        "audio_clasificador": audio_clf_resultado,
        "tipo_error": tipo_error if (confianza_suficiente and veredicto_final == "a_practicar") else None,
        "palabra_distinta": palabra_distinta and veredicto_final == "a_practicar",
        "class_name": clase_predicha,
        "prob": top_prob,
        "detected": detected,
        "total": total,
        "modelo_video": modelo_video,
        "confianza_suficiente": confianza_suficiente,
        "capture_file": capture_file,
        "all_probs": all_probs,
    }
```

def predict

server/main.py:1203-1253

```
@app.post("/predict")
async def predict(
    file: UploadFile = File(None),
    audio: UploadFile = File(None),
    palabra: str = Form(None),
):
    if _state["model_cara"] is None:
        raise HTTPException(status_code=503, detail="El servidor todavía está cargando el modelo.")

    audio_path = None
    if audio is not None:
        with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp_audio:
            shutil.copyfileobj(audio.file, tmp_audio)
            audio_path = tmp_audio.name

    if file is None:
        if audio_path is None:
            raise HTTPException(status_code=400, detail="Hace falta mandar audio o video.")
        try:
            return _run_audio_only_pipeline(audio_path, palabra_esperada=palabra)
        finally:
            try:
                os.remove(audio_path)
            except OSError:
                pass

    landmarker = _state["landmarker"]
    model = _state["model_cara"]
    class_names = _state["class_names_cara"]
    max_frames = _state["max_frames_cara"]

    suffix = os.path.splitext(file.filename or "")[1] or ".mp4"
    with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
        shutil.copyfileobj(file.file, tmp)
        tmp_path = tmp.name

    try:
        return _run_prediction_pipeline(
            tmp_path, suffix, landmarker, model, class_names, max_frames,
```

```
            audio_path=audio_path, palabra_esperada=palabra,
        )
    finally:
        try:
            os.remove(tmp_path)
        except OSError:
            pass
        if audio_path is not None:
            try:
                os.remove(audio_path)
            except OSError:
                pass
```

def _distancia_frase

server/main.py:1259-1269

```
def _distancia_frase(frase, resultado):
    try:
        import audio_pronunciation as _ap
        _ids, usados = _ap._phoneme_to_token_ids(_ap.texto_a_fonemas(frase), _state["audio_vocab"])
        reconocidos = resultado["reconocido_libre"].split()
        if not usados:
            return None
        d = _ap._distancia_edicion(usados, reconocidos)
        return d / max(len(usados), len(reconocidos), 1)
    except Exception:
        return None
```

def predict_frase

server/main.py:1272-1372

```
@app.post("/predict_frase")
async def predict_frase(
    audio: UploadFile = File(...),
    frase: str = Form(...),
    palabra: str = Form(...),
):
    if _state["audio_model"] is None:
        raise HTTPException(status_code=503, detail="El servidor todavía está cargando el modelo de audio.")

    with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp_audio:
        shutil.copyfileobj(audio.file, tmp_audio)
        audio_path = tmp_audio.name

    audio_recortado_path = None
    try:
        resultado = _score_word_in_sentence(audio_path, frase, palabra)
        if resultado is None:
            return {"valid": False, "reason": "no se pudo analizar el audio -- probá de nuevo"}

        audio_recortado_path = resultado.get("audio_recortado_path")

        if not resultado.get("frase_completa_dicha", True):
            _guardar_debug_frase(audio_path, audio_recortado_path, palabra, "incompleta")
            return {
                "valid": False,
                "frase_incompleta": True,
                "reason": "decí toda la frase, despacito -- ¡vos podés!",
            }

        dist_frase = _distancia_frase(frase, resultado)
        if dist_frase is None:
            frase_ok = resultado.get("resto_de_la_frase_ok", True)
        else:
            frase_ok = dist_frase <= FRASE_DISTANCIA_MAXIMA
        if not frase_ok:
            _guardar_debug_frase(audio_path, audio_recortado_path, palabra,
                                 f"distinta_d{'' if dist_frase is None else round(dist_frase, 2)}")
            return {
                "valid": False,
                "frase_distinta": True,
                "reason": "repetí la frase de arriba tal cual -- ¡dale, de nuevo!",
            }

        clf_resultado = _classify_audio(audio_recortado_path, palabra) if audio_recortado_path else None

        gop_ok = len(resultado["sospechosos"]) == 0 and resultado["palabra_reconocida"]

        clf_confiable = clf_resultado is not None and clf_resultado["prob"] >= AUDIO_MIN_CONFIDENCE
        if clf_confiable:
            veredicto_final = "bien" if clf_resultado["es_correcto"] else "a_practicar"
```

```
            fuente = "clasificador"
        else:
            veredicto_final = "bien" if gop_ok else "a_practicar"
            fuente = "gop"
        tipo_error = resultado.get("tipo_error_audio") if veredicto_final == "a_practicar" else None

        if veredicto_final == "a_practicar" or not clf_confiable:
            _prob_dbg = f"{clf_resultado['prob']:.2f}" if clf_resultado else "sinclf"
            _guardar_debug_frase(audio_path, audio_recortado_path, palabra,
                                 f"{veredicto_final}_{fuente}_clf{_prob_dbg}")

        prob_str = f"{clf_resultado['prob']:.2f}" if clf_resultado else "?"
        clase_str = clf_resultado["clase_predicha"] if clf_resultado else "sin dato"
        print(f"[predict-frase] palabra={palabra!r} en frase={frase!r} | "
              f"GOP {'bien' if gop_ok else 'sospechoso'} | "
              f"clasificador {clase_str} prob={prob_str} | decide={fuente} -> {veredicto_final}")

        db.save_attempt(
            class_name=f"{palabra}_en_frase", palabra=palabra, correcta=(veredicto_final == "bien"),
            tipo_error=tipo_error, prob=clf_resultado["prob"] if clf_resultado else (0.0 if gop_ok else 1.0),
        )

        return {
            "valid": True,
            "palabra": palabra,
            "frase": frase,
            "correcta": veredicto_final == "bien",
            "veredicto_final": veredicto_final,
            "decidido_por": fuente,
            "audio": {
                "fonemas": resultado["fonemas"],
                "scores": resultado["scores"],
                "sospechosos": resultado["sospechosos"],
                "reconocido_libre": resultado["reconocido_libre"],
                "sonido_reconocido": resultado["sonido_reconocido"],
                "palabra_reconocida": resultado["palabra_reconocida"],
            },
            "audio_clasificador": clf_resultado,
            "tipo_error": tipo_error,
            "confianza_suficiente": True,
        }
    finally:
        if audio_recortado_path:
            try:
                os.remove(audio_recortado_path)
            except OSError:
                pass
        try:
            os.remove(audio_path)
        except OSError:
            pass
```

def _ciclos_pataka

server/main.py:1375-1384

```
def _ciclos_pataka(tokens):
    siguiente = {"p": "t", "t": "k", "k": "p"}
    esperado, ciclos = "p", 0
    for tok in tokens:
        base = tok[:1]
        if base == esperado:
            if esperado == "k":
                ciclos += 1
            esperado = siguiente[esperado]
    return ciclos
```

def _palabras_flojas

server/main.py:1391-1410

```
def _palabras_flojas(texto, resultado):
    try:
        import audio_pronunciation as _ap
        vocab = _state["audio_vocab"]
        palabras, pos, malas = texto.split(), 0, []
        for pal in palabras:
            _ids, usados = _ap._phoneme_to_token_ids(_ap.texto_a_fonemas(pal), vocab)
            n = len(usados)
            if n == 0:
                continue
            if pal.lower() not in _PALABRAS_DE_RELLENO:
                scores = resultado["scores"][pos:pos + n]
```

```
                if any(sc is None or sc < _UMBRAL_SONIDO_MAL for sc in scores):
                    malas.append(pal)
            pos += n
        if pos != len(resultado["fonemas"]):
            return None
        return malas
    except Exception:
        return None
```

def ejercicio_resultado

server/main.py:1413-1434

```
@app.post("/ejercicio_resultado")
def ejercicio_resultado(
    ejercicio_id: str = Form(...),
    tipo: str = Form(...),
    correcto: bool = Form(...),
    fuerza: int = Form(None),
    aguante: int = Form(None),
    parejo: int = Form(None),
    voz: int = Form(None),
    porcentaje_ok: float = Form(None),
    tipo_error: str = Form(None),
):
    if tipo not in ("praxia", "soplo", "voz"):
        raise HTTPException(status_code=400, detail="tipo tiene que ser praxia, soplo o voz.")
    for nombre, valor in (("fuerza", fuerza), ("aguante", aguante), ("parejo", parejo), ("voz", voz)):
        if valor is not None and not 1 <= valor <= 3:
            raise HTTPException(status_code=400, detail=f"{nombre} va de 1 a 3 estrellas.")
    guardado = db.save_ejercicio(
        ejercicio_id=ejercicio_id[:40], tipo=tipo, correcto=correcto, fuerza=fuerza, aguante=aguante,
        parejo=parejo, voz=voz, porcentaje_ok=porcentaje_ok, tipo_error=tipo_error,
    )
    return {"guardado": guardado}
```

def ejercicios_resumen

server/main.py:1437-1440

```
@app.get("/ejercicios/resumen")
def ejercicios_resumen():
    filas = db.get_ejercicios_resumen()
    return {"ejercicios": filas if filas is not None else [], "base_de_datos": filas is not None}
```

def ejercicio_voz

server/main.py:1443-1512

```
@app.post("/ejercicio_voz")
async def ejercicio_voz(
    audio: UploadFile = File(...),
    texto: str = Form(...),
    tipo: str = Form("trabalenguas"),
):
    if _state["audio_model"] is None:
        raise HTTPException(status_code=503, detail="El servidor todavía está cargando el modelo de audio.")

    with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp_audio:
        shutil.copyfileobj(audio.file, tmp_audio)
        audio_path = tmp_audio.name

    try:
        waveform = load_audio_16k(audio_path)
        pico = float(waveform.abs().max()) if waveform.numel() else 0.0
        if pico < 0.01:
            return {"valid": False, "reason": "no te escuché -- ¡probá de nuevo, más fuerte!"}
        activo = (waveform.abs() > 0.05 * pico).nonzero()
        seg_habla = max(float(activo[-1] - activo[0]) / TARGET_SAMPLE_RATE, 0.5) if len(activo) else 0.5

        resultado = _score_audio_pronunciation(audio_path, texto)
        if resultado is None:
            return {"valid": False, "reason": "no se pudo analizar el audio -- probá de nuevo"}

        scores = resultado["scores"]
        flojos = set(resultado["sospechosos"])
        ok = [i for i, sc in enumerate(scores) if sc is not None and i not in flojos]
        pct_ok = len(ok) / max(len(scores), 1)

        salida = {
            "valid": True,
            "tipo": tipo,
            "porcentaje_ok": round(100 * pct_ok),
```

```
            "palabra_reconocida": resultado["palabra_reconocida"],
            "segundos_de_habla": round(seg_habla, 1),
            "tipo_error": resultado.get("tipo_error_audio"),
        }
        tokens = resultado["reconocido_libre"].split()
        if tipo == "ritmo":
            ciclos = _ciclos_pataka(tokens)
            silabas = sum(1 for t in tokens if t[:1] in ("p", "t", "k"))
            salida.update({
                "ciclos": ciclos,
                "silabas": silabas,
                "silabas_por_segundo": round(silabas / seg_habla, 1),
                "correcta": ciclos >= 3,
            })
        else:
            malas = _palabras_flojas(texto, resultado)
            salida["palabras_flojas"] = malas if malas is not None else []
            salida["correcta"] = (
                pct_ok >= 0.75
                and bool(resultado["palabra_reconocida"])
                and not malas
            )
        _guardar_debug_frase(audio_path, None, f"ej_{tipo}", f"ok{salida['porcentaje_ok']}_{'bien' if salida['correcta'] else 'mal'}")

        print(f"[ejercicio-voz] tipo={tipo} texto={texto!r} | ok={salida['porcentaje_ok']}% "
              f"reconocida={salida['palabra_reconocida']} | correcta={salida['correcta']}"
              + (f" ciclos={salida['ciclos']}" if tipo == "ritmo" else "")
              + (f" flojas={salida.get('palabras_flojas')}" if tipo != "ritmo" else ""))
        return salida
    except Exception as e:
        return {"valid": False, "reason": f"no se pudo analizar el audio ({e}) -- probá de nuevo"}
    finally:
        try:
            os.remove(audio_path)
        except OSError:
            pass
```

Imports, constantes y código suelto

server/main.py:1-1522

```
# Este import va PRIMERO a propósito: si pyarrow (que transformers/pandas cargan más tarde, al abrir
# el modelo de audio) se importa después de torch/cv2/CUDA, en esta PC Windows el proceso muere con
# "heap corruption" (0xC0000374) justo al cargar wav2vec2. Cargándolo antes, el servidor arranca bien.
try:
    import pyarrow.compute  # noqa: F401
except Exception:
    pass

import math
import os
import random
import shutil
import sys
import tempfile
import time
from collections import defaultdict
from datetime import datetime

import av
import cv2
import numpy as np
import torch
import torch.nn as nn
import torchaudio
from fastapi import FastAPI, Form, UploadFile, File, HTTPException
from fastapi.responses import Response
from fastapi.middleware.cors import CORSMiddleware

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

SCRIPTS_DIR = os.path.join(BASE_DIR, "scripts")
if SCRIPTS_DIR not in sys.path:
    sys.path.insert(0, SCRIPTS_DIR)
from audio_pronunciation import (
    EMBEDDING_DIM,
    TARGET_SAMPLE_RATE,
    extract_embedding,
    load_audio_16k,
    load_model as load_audio_model,
    score_pronunciation,
    score_word_in_sentence,
)
```

```
from train_audio_classifier import AudioClassifierHead

import db

AUDIO_CLASSIFIER_PATH = os.path.join(BASE_DIR, "models", "audio_classifier.pth")

SESSIONS_DIR = os.path.join(BASE_DIR, "data", "sesiones_continuas")
ARCHIVADO_SESSIONS_DIR = os.path.join(BASE_DIR, "dataARCHIVADO", "sesiones_continuas")
# [LABIOS DESACTIVADO] MODEL_PATH = os.path.join(BASE_DIR, "models", "best.pth")
MODELO_CARA_PATH = os.path.join(BASE_DIR, "data", "modelos_cara_completa", "best_cara_completa.pth")
# [LABIOS DESACTIVADO] USAR_MODELO_CARA = os.environ.get("FONOKIDS_CARA", "1") != "0"
MIN_DETECCION_CARA = 0.6
OJO_DERECHO_EXTERNO = 36
OJO_IZQUIERDO_EXTERNO = 45
GOAL_PER_WORD = 500

MIN_FRAMES_VALID = 15
MIN_DETECTION_RATE_VALID = 0.6

MAX_FRAME_WIDTH = 640

AUDIO_MIN_CONFIDENCE = 0.60

AUDIO_CONFIRMA_CON_GOP = 0.30
ACOUSTIC_MATCH_THRESHOLD = 0.80
VISUAL_MATCH_THRESHOLD = 0.75

SUFIJO_CORRECTO = "_correcto"
MARCADOR_INCORRECTO = "_incorrecto"

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

LIP_INDICES = list(range(48, 68))
MOUTH_RIGHT_CORNER = 48
MOUTH_LEFT_CORNER = 54
SCALE_EPSILON = 1e-6
HRNET_BATCH_CHUNK_SIZE = 4

OUTLIER_MAX_JUMP = 0.15

SMOOTHING_ALPHA = 0.4

app = FastAPI(title="FonoKids API")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

_state = {
    "model": None, "landmarker": None, "class_names": None, "max_frames": None,
    "audio_model": None, "audio_feature_extractor": None, "audio_vocab": None,
    "audio_clf": None, "audio_clf_classes": None, "audio_clf_mean": None, "audio_clf_std": None,
    "model_cara": None, "class_names_cara": None, "max_frames_cara": None,
}

VIDEO_EXTENSIONS = (".avi", ".webm", ".mp4", ".mov")

_reference_cache = {}

DEBUG_FRASES_DIR = os.path.join(BASE_DIR, "data", "debug_frases")
DEBUG_FRASES_MAX_ARCHIVOS = 80

PREDICT_HARD_TIMEOUT_S = 90

FRASE_DISTANCIA_MAXIMA = 0.48

_PALABRAS_DE_RELLENO = {"en", "un", "una", "el", "la", "los", "las", "de", "del", "y", "e", "o", "a", "al", "con", "por"}
_UMBRAL_SONIDO_MAL = -4.0

if __name__ == "__main__":
    import uvicorn

    pid_file = os.path.join(BASE_DIR, "server", ".server.pid")
    with open(pid_file, "w") as f:
        f.write(str(os.getpid()))

    uvicorn.run(app, host="127.0.0.1", port=8000)
```

6. Base de datos (server/db.py)

def _get_pool

server/db.py:18-34

```
def _get_pool():
    global _pool
    if _pool is not None:
        return _pool
    if mysql is None:
        return None
    try:
        _pool = pooling.MySQLConnectionPool(
            pool_name="fonokids_pool",
            pool_size=5,
            host=DB_HOST, port=DB_PORT, user=DB_USER, password=DB_PASSWORD, database=DB_NAME,
        )
    except Exception as e:
        print(f"[AVISO] No se pudo conectar a MySQL ({e}) -- Logros/Para practicar solo van a "
              "usar los archivos, sin guardar en base de datos.")
        _pool = None
    return _pool
```

def init_db

server/db.py:37-85

```
def init_db():
    pool = _get_pool()
    if pool is None:
        return
    try:
        conn = pool.get_connection()
        cur = conn.cursor()
        cur.execute(
            """
            CREATE TABLE IF NOT EXISTS practice_attempts (
                id INT AUTO_INCREMENT PRIMARY KEY,
                class_name VARCHAR(150) NOT NULL,
                palabra VARCHAR(100) NOT NULL,
                correcta TINYINT(1) NOT NULL,
                tipo_error VARCHAR(50) NULL,
                persona VARCHAR(100) NOT NULL DEFAULT 'web',
                prob FLOAT NOT NULL,
                created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
                INDEX idx_palabra (palabra),
                INDEX idx_created_at (created_at)
            ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
            """
        )
        cur.execute(
            """
            CREATE TABLE IF NOT EXISTS ejercicio_intentos (
                id INT AUTO_INCREMENT PRIMARY KEY,
                ejercicio_id VARCHAR(40) NOT NULL,
                tipo VARCHAR(20) NOT NULL,
                correcto TINYINT(1) NOT NULL,
                fuerza TINYINT NULL,
                aguante TINYINT NULL,
                parejo TINYINT NULL,
                voz TINYINT NULL,
                porcentaje_ok FLOAT NULL,
                tipo_error VARCHAR(50) NULL,
                persona VARCHAR(100) NOT NULL DEFAULT 'web',
                created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
                INDEX idx_ejercicio (ejercicio_id),
                INDEX idx_created_at (created_at)
            ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
            """
        )
        conn.commit()
        cur.close()
        conn.close()
        print("[db] Conectado a MySQL -- Logros/Para practicar quedan guardados en la base.")
    except Exception as e:
        print(f"[AVISO] No se pudo inicializar la tabla en MySQL ({e}).")
```

def save_attempt

server/db.py:88-104

```
def save_attempt(class_name, palabra, correcta, tipo_error, prob, persona="web"):
    pool = _get_pool()
    if pool is None:
        return
    try:
        conn = pool.get_connection()
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO practice_attempts (class_name, palabra, correcta, tipo_error, persona, prob) "
            "VALUES (%s, %s, %s, %s, %s, %s)",
            (class_name, palabra, 1 if correcta else 0, tipo_error, persona, float(prob)),
        )
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[AVISO] No se pudo guardar el intento en MySQL ({e}).")
```

def save_attempt_at

server/db.py:107-126

```
def save_attempt_at(class_name, palabra, correcta, tipo_error, prob, created_at_ts, persona="web"):
    import datetime

    pool = _get_pool()
    if pool is None:
        return
    try:
        conn = pool.get_connection()
        cur = conn.cursor()
        created_at = datetime.datetime.fromtimestamp(created_at_ts)
        cur.execute(
            "INSERT INTO practice_attempts (class_name, palabra, correcta, tipo_error, persona, prob, created_at) "
            "VALUES (%s, %s, %s, %s, %s, %s, %s)",
            (class_name, palabra, 1 if correcta else 0, tipo_error, persona, float(prob), created_at),
        )
        conn.commit()
        cur.close()
        conn.close()
    except Exception as e:
        print(f"[AVISO] No se pudo guardar el intento migrado en MySQL ({e}).")
```

def get_recordings_grouped

server/db.py:129-162

```
def get_recordings_grouped():
    pool = _get_pool()
    if pool is None:
        return None
    try:
        conn = pool.get_connection()
        cur = conn.cursor(dictionary=True)
        cur.execute(
            """
            SELECT
                DATE(created_at) AS date,
                DATE_FORMAT(MAX(created_at), '%H:%i') AS time,
                class_name,
                palabra AS word,
                correcta,
                tipo_error,
                persona AS person,
                COUNT(*) AS count
            FROM practice_attempts
            GROUP BY DATE(created_at), class_name, persona
            ORDER BY MAX(created_at) DESC
            """
        )
        rows = cur.fetchall()
        cur.close()
        conn.close()
        for r in rows:
            r["date"] = str(r["date"])
            r["correcta"] = bool(r["correcta"])
            r["thumbnail_file"] = None
        return rows
    except Exception as e:
```

```
        print(f"[AVISO] No se pudo leer de MySQL ({e}).")
        return None
```

def save_ejercicio

server/db.py:165-186

```
def save_ejercicio(ejercicio_id, tipo, correcto, fuerza=None, aguante=None, parejo=None, voz=None,
                   porcentaje_ok=None, tipo_error=None, persona="web"):
    pool = _get_pool()
    if pool is None:
        return False
    try:
        conn = pool.get_connection()
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO ejercicio_intentos "
            "(ejercicio_id, tipo, correcto, fuerza, aguante, parejo, voz, porcentaje_ok, tipo_error, persona) "
            "VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)",
            (ejercicio_id, tipo, 1 if correcto else 0, fuerza, aguante, parejo, voz,
             None if porcentaje_ok is None else float(porcentaje_ok), tipo_error, persona),
        )
        conn.commit()
        cur.close()
        conn.close()
        return True
    except Exception as e:
        print(f"[AVISO] No se pudo guardar el ejercicio en MySQL ({e}).")
        return False
```

def get_ejercicios_resumen

server/db.py:189-223

```
def get_ejercicios_resumen():
    pool = _get_pool()
    if pool is None:
        return None
    try:
        conn = pool.get_connection()
        cur = conn.cursor(dictionary=True)
        cur.execute(
            """
            SELECT
                ejercicio_id,
                tipo,
                COUNT(*) AS intentos,
                SUM(correcto) AS bien,
                ROUND(AVG(fuerza), 1) AS fuerza_media,
                ROUND(AVG(aguante), 1) AS aguante_medio,
                ROUND(AVG(parejo), 1) AS parejo_medio,
                ROUND(AVG(voz), 1) AS voz_media,
                DATE_FORMAT(MAX(created_at), '%Y-%m-%d %H:%i') AS ultimo
            FROM ejercicio_intentos
            GROUP BY ejercicio_id, tipo
            ORDER BY MAX(created_at) DESC
            """
        )
        rows = cur.fetchall()
        cur.close()
        conn.close()
        for r in rows:
            r["bien"] = int(r["bien"] or 0)
            for k in ("fuerza_media", "aguante_medio", "parejo_medio", "voz_media"):
                r[k] = None if r[k] is None else float(r[k])
        return rows
    except Exception as e:
        print(f"[AVISO] No se pudo leer los ejercicios de MySQL ({e}).")
        return None
```

Imports, constantes y código suelto

server/db.py:1-15

```
import os

try:
    import mysql.connector
    from mysql.connector import pooling
except ImportError:
    mysql = None

DB_HOST = os.environ.get("FONOKIDS_DB_HOST", "127.0.0.1")
```

```
DB_PORT = int(os.environ.get("FONOKIDS_DB_PORT", "3306"))
DB_USER = os.environ.get("FONOKIDS_DB_USER", "root")
DB_PASSWORD = os.environ.get("FONOKIDS_DB_PASSWORD", "")
DB_NAME = os.environ.get("FONOKIDS_DB_NAME", "fonokids")

_pool = None
```

7. Web · ejercicios y esfuerzo (web/src/ejercicios.js)

function media

web/src/ejercicios.js:73-73

```
const media = (v) => v.reduce((a, x) => a + x, 0) / Math.max(v.length, 1);
```

function nivel

web/src/ejercicios.js:74-74

```
const nivel = (valor, cortes) => (valor >= cortes[1] ? 3 : valor >= cortes[0] ? 2 : 1);
```

function esfuerzoPraxia

web/src/ejercicios.js:83-95

```
/// Praxias. `reps`: una entrada por repetición con `fuerza` (intensidad media
/// durante los 3 s, 1 = su máximo) y `firme` (fracción del tiempo en que llegó
/// al 75 % de su máximo o más).
export function esfuerzoPraxia(reps) {
  if (!reps?.length) return null;
  const fuerzas = reps.map((r) => r.fuerza);
  const alto = Math.max(...fuerzas);
  const bajo = Math.min(...fuerzas);
  return {
    fuerza: nivel(media(fuerzas), [0.65, 0.9]),
    aguante: nivel(media(reps.map((r) => r.firme)), [0.5, 0.85]),
    parejo: nivel(alto > 0 ? 1 - (alto - bajo) / alto : 0, [0.7, 0.9]),
  };
```

function esfuerzoSoplo

web/src/ejercicios.js:98-106

```
/// Soplo. No hay máximo personal (la calibración solo mide el ruido de fondo),
/// así que la fuerza se compara contra el umbral de soplo: `rmsMedio` y
/// `umbral` salen del volumen del micrófono; `firme` es la fracción del soplo
/// con volumen de al menos 1,5 veces el umbral.
export function esfuerzoSoplo(rmsMedio, umbral, firme) {
  return {
    fuerza: nivel(rmsMedio / umbral, [1.5, 2.5]),
    aguante: nivel(firme, [0.5, 0.85]),
  };
```

function esfuerzoVoz

web/src/ejercicios.js:109-126

```
/// Práctica de palabras y frases. Mide qué tan fuerte y clara se oyó la voz
/// frente al ruido de fondo de ESA grabación: el volumen de los tramos más
/// fuertes dividido por el de los más tranquilos (percentil 90 sobre percentil
/// 10 del volumen de cada bloque de audio). No usa un volumen absoluto porque
/// depende de la ganancia del micrófono. `chunks`: bloques Float32 del
/// grabador. Devuelve null si el audio es demasiado corto para medir.
export function esfuerzoVoz(chunks) {
  if (!chunks || chunks.length < 6) return null;
  const rms = chunks.map((c) => {
    let s = 0;
    for (let i = 0; i < c.length; i++) s += c[i] * c[i];
    return Math.sqrt(s / Math.max(c.length, 1));
  }).sort((a, b) => a - b);
  const pct = (p) => rms[Math.floor(p * (rms.length - 1))];
  const voz = pct(0.9);
  const ruido = pct(0.1);
  if (voz < 0.005) return { voz: 1 }; // casi no se oyó nada
  return { voz: nivel(voz / Math.max(ruido, 1e-4), [3, 8]) };
```

Imports, constantes y código suelto

web/src/ejercicios.js

```
// Ejercicios de calentamiento (versión de PRUEBA). Cada ficha dice qué se le
// muestra al chico, cómo se evalúa y cuándo se da por pasado.
//
// Tipos:
//   praxia -> cámara en el navegador (MediaPipe, blendshapes). Se mide un
//             valor de la cara; tiene que superar `deltas` (uno por señal medida; alcanza con una) por encima del
//             reposo del propio chico (calibración de 2 s) y mantenerse
//             HOLD_MS, `repeticiones` veces.
```

```
//   soplo  -> micrófono en el navegador. Se mide el volumen sostenido.
//   voz    -> graba y manda el audio a POST /ejercicio_voz del servidor.
//             tipo "ritmo" (cuenta vueltas p-t-k) o "trabalenguas" (GOP).
//
// Los umbrales (`delta`) son valores iniciales para probar, a calibrar con
// varias personas -- y el contenido tiene que revisarlo un especialista.

export const HOLD_MS = 3000; // cuánto se mantiene el gesto para contar 1 repetición
export const REPETICIONES = 3;

export const EJERCICIOS = [
  {
    id: "sonrisa", tipo: "praxia", icono: "😁", titulo: "Sonrisa grande",
    instruccion: "¡Sonríe lo más grande que puedas y mantenlo!", medida: "sonrisa", deltas: [0.3],
  },
  {
    id: "beso", tipo: "praxia", icono: "😗", titulo: "Beso",
    instruccion: "¡Haz un besito con los labios hacia adelante y mantenlo!", medida: "beso", deltas: [0.3],
  },
  {
    id: "boca", tipo: "praxia", icono: "😮", titulo: "Boca abierta",
    instruccion: "¡Abre la boca bien grande, como un bostezo, y mantenlo!", medida: "boca", deltas: [0.35],
  },
  {
    id: "mejillas", tipo: "praxia", icono: "🐡", titulo: "Mejillas infladas",
    instruccion: "¡Infla las mejillas como un globo y mantenlo!", medida: "mejillas", deltas: [0.08, 0.04],
  },
  {
    id: "vela", tipo: "soplo", icono: "🕯️", titulo: "Soplar la vela",
    instruccion: "Toma aire y sopla suave y largo para apagar la vela.",
  },
  {
    id: "pataka", tipo: "voz", modo: "ritmo", icono: "🐷", titulo: "Pa-ta-ka rápido",
    instruccion: "Repite «pa-ta-ka» seguido y rápido, ¡3 veces o más!",
    texto: "pa ta ka pa ta ka pa ta ka", mostrar: "PA-TA-KA  PA-TA-KA  PA-TA-KA",
  },
  {
    id: "tigres", tipo: "voz", modo: "trabalenguas", icono: "🐯", titulo: "Los tres tigres",
    instruccion: "Dilo despacito y claro.",
    texto: "tres tristes tigres tragaban trigo en un trigal",
    mostrar: "Tres tristes tigres tragaban trigo en un trigal",
  },
  {
    id: "clavito", tipo: "voz", modo: "trabalenguas", icono: "🔨", titulo: "Pablito y el clavito",
    instruccion: "Dilo despacito y claro.",
    texto: "pablito clavó un clavito", mostrar: "Pablito clavó un clavito",
  },
  {
    id: "erre", tipo: "voz", modo: "trabalenguas", icono: "🚂", titulo: "Los carros del ferrocarril",
    instruccion: "Dilo despacito y claro.",
    texto: "rápido ruedan los carros del ferrocarril", mostrar: "Rápido ruedan los carros del ferrocarril",
  },
];

export const ETIQUETA_TIPO = { praxia: "Cara", soplo: "Soplo", voz: "Voz" };

// Estimación del esfuerzo (versión de PRUEBA, sin validar con niños ni con un
// especialista). No mide el esfuerzo muscular real: estima qué tan fuerte, qué
// tan sostenido y qué tan parejo fue el gesto respecto al MÁXIMO de esa misma
// persona (el que se mide en el paso 2 de la praxia). Se muestra al chico solo
// como estrellas (1 a 3), nunca como porcentaje. Los cortes son valores
// iniciales a calibrar.
const FUERTE = 0.75; // un instante cuenta como «firme» si llega al 75 % de su máximo

export const ETIQUETAS_ESFUERZO = {
  fuerza: "💪 Fuerza",
  aguante: "⏱️ Aguante",
  parejo: "🔁 Repeticiones parejas",
  voz: "🗣️ Voz fuerte y clara",
};

}

}

}

export const SOPLO_FIRME = 1.5;
export const GESTO_FIRME = FUERTE;
```

8. Web · llamadas al servidor (web/src/api.js)

function normalize

web/src/api.js:9-12

```
function normalize(url) {
  const u = url.trim();
  return u.endsWith("/") ? u.slice(0, -1) : u;
}
```

function getServerUrl

web/src/api.js:14-16

```
export function getServerUrl() {
  return localStorage.getItem(STORAGE_KEY) || DEFAULT_URL;
}
```

function thumbnailUrl

web/src/api.js:18-25

```
/// URL de la foto real (frame del medio del clip) para una captura --
/// GET /dataset/thumbnail en server/main.py. Puede devolver 404 si el clip
/// es viejo (dataset de entrenamiento) o no tiene thumbnail_file -- la UI
/// debe manejar el error de carga y caer al placeholder.
export function thumbnailUrl(baseUrl, word, filename) {
  const params = new URLSearchParams({ word, filename });
  return `${normalize(baseUrl)}/dataset/thumbnail?${params}`;
}
```

function setServerUrl

web/src/api.js:27-29

```
export function setServerUrl(url) {
  localStorage.setItem(STORAGE_KEY, url.trim());
}
```

function withTimeout

web/src/api.js:31-44

```
async function withTimeout(promise, ms, timeoutMessage) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), ms);
  try {
    return await promise(controller.signal);
  } catch (e) {
    if (e.name === "AbortError") {
      throw new ApiError(timeoutMessage);
    }
    throw e;
  } finally {
    clearTimeout(timer);
  }
}
```

function checkHealth

web/src/api.js:46-57

```
export async function checkHealth(baseUrl) {
  try {
    const res = await withTimeout(
      (signal) => fetch(`${normalize(baseUrl)}/health`, { signal }),
      5000,
      "timeout"
    );
    return res.ok;
  } catch {
    return false;
  }
}
```

function getHealth

web/src/api.js:59-75

```
/// Igual que checkHealth pero devuelve el cuerpo entero (incluye "classes",
/// la lista de clases que reconoce el modelo cargado) en vez de solo
/// true/false -- lo usa el selector de palabra objetivo en Practicar.
/// null si el server no responde (mismo criterio que checkHealth).
export async function getHealth(baseUrl) {
  try {
    const res = await withTimeout(
      (signal) => fetch(`${normalize(baseUrl)}/health`, { signal }),
      5000,
      "timeout"
    );
    if (!res.ok) return null;
    return await res.json();
  } catch {
    return null;
  }
}
```

function getReferenceLandmarks

web/src/api.js:77-92

```
/// Secuencia real de puntos de labios (20 puntos x,y por frame) de un clip
/// "<palabra>_correcto" -- para animar la boquita de referencia. Null si
/// todavía no hay ningún clip correcto grabado de esa palabra.
export async function getReferenceLandmarks(baseUrl, palabra) {
  try {
    const res = await withTimeout(
      (signal) => fetch(`${normalize(baseUrl)}/reference/${encodeURIComponent(palabra)}`, { signal }),
      20000,
      "timeout"
    );
    if (!res.ok) return null;
    return await res.json();
  } catch {
    return null;
  }
}
```

function wordsFromClasses

web/src/api.js:94-106

```
/// De la lista de clases del modelo ("perro_correcto", "perro_incorrecto_
/// lambdacismo", ...) saca las palabras objetivo ÚNICAS ("perro", ...),
/// mismo criterio que _parse_clase_predicha() en server/main.py -- corta
/// en "_correcto" o en "_incorrecto" (lo que aparezca primero).
export function wordsFromClasses(classNames) {
  if (!Array.isArray(classNames)) return [];
  const words = new Set();
  for (const c of classNames) {
    const idx = c.endsWith("_correcto") ? c.length - "_correcto".length : c.indexOf("_incorrecto");
    words.add(idx > 0 ? c.slice(0, idx) : c);
  }
  return [...words].sort();
}
```

function getDatasetStats

web/src/api.js:115-125

```
export async function getDatasetStats(baseUrl) {
  const res = await withTimeout(
    (signal) => fetch(`${normalize(baseUrl)}/dataset/stats`, { signal }),
    DATASET_TIMEOUT_MS,
    "El servidor tardó demasiado en responder."
  ).catch((e) => {
    throw new ApiError(e.message || `No se pudo conectar al servidor (${baseUrl}).`);
  });
  if (!res.ok) throw new ApiError(`El servidor respondió con error ${res.status}.`);
  return res.json();
}
```

function getRecordings

web/src/api.js:127-137

```
export async function getRecordings(baseUrl) {
  const res = await withTimeout(
    (signal) => fetch(`${normalize(baseUrl)}/dataset/recordings`, { signal }),
```

```
    DATASET_TIMEOUT_MS,
    "El servidor tardó demasiado en responder."
  ).catch((e) => {
    throw new ApiError(e.message || `No se pudo conectar al servidor (${baseUrl}).`);
  });
  if (!res.ok) throw new ApiError(`El servidor respondió con error ${res.status}.`);
  return res.json();
}
```

function waitForServerBack

web/src/api.js:139-150

```
async function waitForServerBack(baseUrl, maxWaitMs = 60000) {
  // El servidor se reinicia solo cuando /predict se cuelga (ver server/
  // main.py, PREDICT_HARD_TIMEOUT_S) -- típicamente vuelve a responder en
  // 20-30s. Sondeamos /health hasta que vuelva, en vez de reintentar a
  // ciegas contra un servidor que todavía se está reiniciando.
  const start = Date.now();
  while (Date.now() - start < maxWaitMs) {
    if (await checkHealth(baseUrl)) return true;
    await new Promise((r) => setTimeout(r, 2500));
  }
  return false;
}
```

function predictOnce

web/src/api.js:152-201

```
async function predictOnce(baseUrl, videoBlob, filename, audioBlob, palabra) {
  const uri = `${normalize(baseUrl)}/predict`;
  const form = new FormData();
  // El video es OPCIONAL -- modo "solo voz" (sin cámara, ver
  // CameraConsentGate en HomeScreen.jsx): no se manda "file" en absoluto, y
  // el servidor decide el veredicto solo con el audio.
  if (videoBlob) form.append("file", videoBlob, filename);
  // Audio y palabra son OPCIONALES -- si no hay micrófono (o el navegador
  // lo bloqueó), el servidor sigue funcionando solo con el video, igual que
  // antes de agregar el análisis de sonido.
  if (audioBlob) form.append("audio", audioBlob, "clip.wav");
  if (palabra) form.append("palabra", palabra);

  let response;
  try {
    response = await withTimeout(
      (signal) => fetch(uri, { method: "POST", body: form, signal }),
      150000,
      "El servidor tardó demasiado en responder. Puede estar sobrecargado " +
        "(GPU con poca memoria) -- probá de nuevo en unos segundos."
    );
  } catch (e) {
    if (e instanceof ApiError) throw e;
    throw new ApiError(
      `No se pudo conectar al servidor (${baseUrl}). Revisá que esté prendido (python server/main.py).`
    );
  }

  if (!response.ok) {
    // El servidor manda un "detail" específico y útil (ej. "se está
    // reiniciando solo, probá de nuevo en unos segundos" en un 504) --
    // antes se ignoraba y siempre se mostraba un mensaje genérico. Mejor
    // mostrar el real cuando existe.
    let detail = null;
    try {
      const body = await response.json();
      detail = body?.detail || null;
    } catch {
      // Respuesta sin JSON (ej. error crudo del servidor web) -- seguimos
      // con el mensaje genérico de abajo.
    }
    const err = new ApiError(
      detail || `El servidor respondió con error ${response.status}. Revisá que el modelo esté cargado ahí.`
    );
    err.status = response.status;
    throw err;
  }

  return response.json();
}
```

function predictFrase

web/src/api.js:203-252

```
/// Envía el clip grabado (Blob) al servidor y devuelve la predicción.
//
// Reintento automático ante un cuelgue de GPU: el servidor a veces se
// cuelga por un problema intermitente del driver de NVIDIA/CUDA (no del
// código ni del clip -- confirmado probando el MISMO clip varias veces:
// unas anda, otras se cuelga) y se mata/reinicia solo en ~90s (ver
// PREDICT_HARD_TIMEOUT_S en server/main.py). Antes, cuando pasaba eso, el
// usuario tenía que darse cuenta del error y volver a grabar a mano. Ahora,
// si la respuesta es justo ese 504 de "me estoy reiniciando", esperamos a
// que el servidor vuelva a responder /health y reintentamos el MISMO clip
// una sola vez automáticamente -- en la mayoría de los casos ya funciona
// en el segundo intento, sin que el usuario tenga que hacer nada.
/// Evalúa UNA palabra entrenada dentro de una FRASE completa (ver
/// FrasesScreen.jsx y server/main.py POST /predict_frase) -- distinto de
/// predict() de abajo, que evalúa la palabra sola con video+audio. Acá
/// solo se manda audio, nunca video (el modo frase es siempre "solo voz").
export async function predictFrase(baseUrl, audioBlob, frase, palabra) {
  const uri = `${normalize(baseUrl)}/predict_frase`;
  const form = new FormData();
  form.append("audio", audioBlob, "frase.wav");
  form.append("frase", frase);
  form.append("palabra", palabra);

  let response;
  try {
    response = await withTimeout(
      (signal) => fetch(uri, { method: "POST", body: form, signal }),
      60000,
      "El servidor tardó demasiado en responder -- probá de nuevo."
    );
  } catch (e) {
    if (e instanceof ApiError) throw e;
    throw new ApiError(
      `No se pudo conectar al servidor (${baseUrl}). Revisá que esté prendido.`
    );
  }

  if (!response.ok) {
    let detail = null;
    try {
      const body = await response.json();
      detail = body?.detail || null;
    } catch {
      // sin JSON -- mensaje genérico de abajo
    }
    throw new ApiError(detail || `El servidor respondió con error ${response.status}.`);
  }

  return response.json();
}
```

function predict

web/src/api.js:254-271

```
export async function predict(
  baseUrl, videoBlob, filename = "clip.webm", onRetrying = null, audioBlob = null, palabra = null
) {
  try {
    return await predictOnce(baseUrl, videoBlob, filename, audioBlob, palabra);
  } catch (e) {
    if (e.status !== 504) throw e;

    if (onRetrying) onRetrying();
    const back = await waitForServerBack(baseUrl);
    if (!back) {
      throw new ApiError(
        "El servidor se colgó y todavía no volvió a responder -- probá de nuevo en un ratito."
      );
    }
    return await predictOnce(baseUrl, videoBlob, filename, audioBlob, palabra);
  }
}
```

function ejercicioVoz

web/src/api.js:273-308

```
/// Ejercicios de voz (prueba) -- POST /ejercicio_voz en server/main.py.
/// tipo: "ritmo" (pa-ta-ka) o "trabalenguas".
```

```
export async function ejercicioVoz(baseUrl, audioBlob, texto, tipo) {
  const uri = `${normalize(baseUrl)}/ejercicio_voz`;
  const form = new FormData();
  form.append("audio", audioBlob, "ejercicio.wav");
  form.append("texto", texto);
  form.append("tipo", tipo);

  let response;
  try {
    response = await withTimeout(
      (signal) => fetch(uri, { method: "POST", body: form, signal }),
      60000,
      "El servidor tardó demasiado en responder -- probá de nuevo."
    );
  } catch (e) {
    if (e instanceof ApiError) throw e;
    throw new ApiError(
      `No se pudo conectar al servidor (${baseUrl}). Revisá que esté prendido.`
    );
  }

  if (!response.ok) {
    let detail = null;
    try {
      const body = await response.json();
      detail = body?.detail || null;
    } catch {
      // sin JSON -- mensaje genérico de abajo
    }
    throw new ApiError(detail || `El servidor respondió con error ${response.status}.`);
  }

  return response.json();
}
```

function guardarEjercicio

web/src/api.js:310-327

```
/// Guarda en la base (tabla ejercicio_intentos) cómo le fue al niño en un ejercicio.
/// No molesta al niño si falla: devuelve false y listo.
export async function guardarEjercicio(baseUrl, datos) {
  const form = new FormData();
  for (const [k, v] of Object.entries(datos)) {
    if (v !== null && v !== undefined) form.append(k, String(v));
  }
  try {
    const res = await withTimeout(
      (signal) => fetch(`${normalize(baseUrl)}/ejercicio_resultado`, { method: "POST", body: form, signal }),
      8000,
      "timeout"
    );
    return res.ok;
  } catch {
    return false;
  }
}
```

Imports, constantes y código suelto

web/src/api.js

```
// Cliente HTTP para server/main.py -- equivalente web de api_client.dart.
const STORAGE_KEY = "speakshadow_server_url";
// server/main.py corre en la MISMA máquina que sirve esta página ("npm run
// dev" y "python server/main.py" en la misma PC, ambos en localhost).
export const DEFAULT_URL = `${window.location.protocol}//${window.location.hostname}:8000`;

export class ApiError extends Error {}

// 20s en vez de 5s -- estas consultas son livianas (listar archivos), pero
// comparten servidor con /predict: si justo hay una predicción corriendo,
// pueden demorar un poco más de lo normal en contestar. Con solo 5s
// cualquier pestañeo hacía que la app cayera a datos de muestra y pareciera
// que "cambiaban solas" entre reales y de muestra.
const DATASET_TIMEOUT_MS = 20000;
```

9. Web · grabador de audio (web/src/audioRecorder.js)

function encodeWav

web/src/audioRecorder.js:9-26

```
function encodeWav(chunks, sampleRate) {
  let totalLength = 0;
  for (const c of chunks) totalLength += c.length;

  const pcm16 = new Int16Array(totalLength);
  let offset = 0;
  for (const chunk of chunks) {
    for (let i = 0; i < chunk.length; i++) {
      const s = Math.max(-1, Math.min(1, chunk[i]));
      pcm16[offset++] = s < 0 ? s * 0x8000 : s * 0x7fff;
    }
  }

  const buffer = new ArrayBuffer(44 + pcm16.length * 2);
  const view = new DataView(buffer);
  const writeStr = (pos, str) => {
    for (let i = 0; i < str.length; i++) view.setUint8(pos + i, str.charCodeAt(i));
  };
```

function createAudioRecorder

web/src/audioRecorder.js:45-66

```
/// Crea un grabador independiente (sin estado de React) -- start(stream)
/// arranca a juntar PCM crudo del track de audio del stream, stop()
/// corta y devuelve el Blob .wav ya armado (o null si no había audio).
export function createAudioRecorder() {
  let ctx = null;
  let source = null;
  let processor = null;
  let chunks = [];
  let ultimoEsfuerzo = null;

  function start(stream) {
    const audioTrack = stream?.getAudioTracks?.()[0];
    if (!audioTrack) return false;
    chunks = [];
    try {
      const AudioCtx = window.AudioContext || window.webkitAudioContext;
      ctx = new AudioCtx();
      source = ctx.createMediaStreamSource(new MediaStream([audioTrack]));
      processor = ctx.createScriptProcessor(4096, 1, 1);
      processor.onaudioprocess = (e) => {
        chunks.push(new Float32Array(e.inputBuffer.getChannelData(0)));
      };
```

Imports, constantes y código suelto

web/src/audioRecorder.js

```
// Grabador de audio -> WAV real (16 bits, mono) usando Web Audio API --
// extraído de HomeScreen.jsx para reusarlo también en FrasesScreen.jsx sin
// duplicar la lógica. El servidor (server/main.py) espera WAV de verdad,
// no el webm/opus que da MediaRecorder por defecto -- por eso se arma el
// header RIFF/WAVE a mano en vez de usar MediaRecorder para el audio.

import { esfuerzoVoz } from "./ejercicios.js";

  writeStr(0, "RIFF");
  view.setUint32(4, 36 + pcm16.length * 2, true);
  writeStr(8, "WAVE");
  writeStr(12, "fmt ");
  view.setUint32(16, 16, true);
  view.setUint16(20, 1, true); // PCM
  view.setUint16(22, 1, true); // mono
  view.setUint32(24, sampleRate, true);
  view.setUint32(28, sampleRate * 2, true);
  view.setUint16(32, 2, true);
  view.setUint16(34, 16, true);
  writeStr(36, "data");
  view.setUint32(40, pcm16.length * 2, true);
  new Int16Array(buffer, 44).set(pcm16);

  return new Blob([buffer], { type: "audio/wav" });
}
```

```
      source.connect(processor);
      processor.connect(ctx.destination);
      return true;
    } catch (e) {
      console.warn("No se pudo iniciar la captura de audio:", e);
      return false;
    }
  }

  function stop() {
    if (!ctx) return null;
    const sampleRate = ctx.sampleRate;
    try {
      processor?.disconnect();
      source?.disconnect();
      ctx.close();
    } catch {
      // no crítico
    }
    const finalChunks = chunks;
    ctx = null;
    source = null;
    processor = null;
    chunks = [];
    ultimoEsfuerzo = esfuerzoVoz(finalChunks);
    if (finalChunks.length === 0) return null;
    return encodeWav(finalChunks, sampleRate);
  }

  /// Estimación de qué tan fuerte y clara se oyó la voz en la última grabación
  /// (o null si fue muy corta); ver esfuerzoVoz en ejercicios.js.
  function esfuerzo() {
    return ultimoEsfuerzo;
  }

  return { start, stop, esfuerzo };
}
```